ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

AI23 (23TNT1), FIT@HCMUS, VNUHCM

Đồ án 1 - Search algorithms

Đề tài: Triển khai, phân tích và so sánh các thuật toán trí tuệ bầy đàn

---

Môn học: Cơ sở trí tuệ nhân tạo - TTNT2023

Sinh viên thực hiện:
Đinh Đức Anh Khoa (23122001)
Nguyễn Lê Hoàng Trung (23122004)
Đinh Đức Tài (23122013)

Giáo viên hướng dẫn:
GS.TS Lê Hoài Bắc
ThS. Lê Nhựt Nam

Ngày 14 tháng 11 năm 2025



Mục lục

1 Giới thiệu **3**

2 Các bài toán tối ưu hóa **4**
2.1 Các benchmark tối ưu hóa liên tục . . . 4
2.1.1 Hàm Sphere . . . 4
2.1.2 Hàm Rastrigin . . . 4
2.2 Bài toán tối ưu hóa rời rạc . . . 5
2.2.1 Travelling salesman problem - TSP . . . 5

3 Trí tuệ bầy đàn cho bài toán tối ưu hóa liên tục **6**
3.1 Cuckoo Search . . . 6
3.1.1 Mô tả thuật toán . . . 6
3.1.2 Tính toán Lévy FLights . . . 6
3.1.3 Cơ chế hoạt động . . . 7
3.1.4 Mã giả . . . 8
3.2 Artificial Bee Colony . . . 8
3.2.1 Mô tả thuật toán . . . 8
3.2.2 Cơ chế hoạt động . . . 9
3.3 Firefly Algorithm . . . 10
3.3.1 Mô tả thuật toán . . . 10
3.3.2 Tính toán độ hấp dẫn . . . 10
3.3.3 Cơ thế hoạt động . . . 11
3.3.4 Mã giả . . . 12

4 Trí tuệ bầy đàn cho bài toán tối ưu hóa rời rạc **12**
4.1 Ant Colony Optimization . . . 12
4.1.1 ACO và tối ưu hóa tổ hợp . . . 12
4.1.2 Cơ chế hoạt động ACO . . . 13
4.1.3 Thuật toán ACO . . . 13
4.2 Particle Swarm Optimization . . . 14
4.2.1 Thuật toán PSO cơ bản . . . 14
4.2.2 Global Best PSO - gbest PSO . . . 15
4.2.3 Local Best PSO - lbest PSO . . . 16
4.2.4 Điều chỉnh thuật toán PSO cho tối ưu hóa rời rạc . . . 16

5 Thuật toán truyền thống cho các bài toán tối ưu **17**
5.1 Hill Climbing cho bài toán tối ưu hóa liên tục . . . 17
5.1.1 Mô tả thuật toán . . . 17
5.1.2 Thành phần cốt lõi . . . 17
5.1.3 Cơ chế hoạt động . . . 18
5.2 Simulated Annealing cho bài toán tối ưu hóa liên tục . . . 18
5.2.1 Mô tả thuật toán . . . 18
5.2.2 Thành phần cốt lõi . . . 19
5.2.3 Cơ chế hoạt động . . . 20
5.3 Tìm kiếm A* cho bài toán tối ưu hóa rời rạc . . . 20

5.3.1 Mô tả thuật toán . . . 20
5.3.2 Sử dụng A* Search cho TSP . . . 21
5.4 Genetic Algorithm cho bài toán tối ưu hóa rời rạc . . . 22
5.4.1 Sử dụng Genetic Algorithm cho TSP . . . 22

6 Thực nghiệm **23**
6.1 Tối ưu hóa liên tục . . . 23
6.1.1 Cuckoo Search - CS . . . 24
6.1.2 Artificial Bee Colony - ABC . . . 26
6.1.3 Firefly Algorithm - FA . . . 28
6.1.4 So sánh giải thuật bầy đàn và giải thuật truyền thống . . . 30
6.2 Tối ưu hóa rời rạc . . . 31
6.2.1 Ant Colony Optimization - ACO . . . 33
6.2.2 Particle Swarm Optimization - PSO . . . 35
6.2.3 Genetic Algorithm - GA . . . 37
6.2.4 A* search . . . 39
6.3 Kết luận . . . 39

7 Kết luận, đánh giá **40**
7.1 Kết luận . . . 40
7.2 Đánh giá . . . 40

1 Giới thiệu

Đây là tài liệu báo cáo cho **Đồ án 1 - Search algorithms**, môn **Cơ sở trí tuệ nhân tạo - TTNT2023**, Khoa Công nghệ thông tin, Trường Đại học Khoa học tự nhiên - Đại học Quốc gia TP.HCM. Chủ đề của đồ án là triển khai, phân tích và so sánh các thuật toán trí tuệ bầy đàn.

Báo cáo được thực hiện bởi nhóm các thành viên:

- Đinh Đức Anh Khoa (23122001)
- Nguyễn Lê Hoàng Trung (23122004)
- Đinh Đức Tài (23122013)

Đường dẫn repository Github: https://github.com/anhKhoa585/AI-Fundamentals-Proj1 [1]

Demo youtube: https://youtu.be/KFdTRbJd9w0 [2]

Bảng phân công nhiệm vụ cho từng thành viên:

Họ và tên	MSSV	Nhiệm vụ
Đinh Đức Anh Khoa	23122001	- Code benchmark tối ưu hóa liên tục: Sphere, Rastrigin - Code giải thuật trí tuệ bầy đàn: CS, ABC, FA - Viết báo cáo Sphere, Rastrigin, CS, ABC, FA. - Thực nghiệm, so sánh các giải thuật tối ưu hóa liên tục.
Nguyễn Lê Hoàng Trung	23122004	- Code thuật toán truyền thống: A*, HC, SA. - Viết báo cáo A*, HC, SA. - Trực quan hóa CS, ABC, FA cho benchmark tối ưu hóa liên tục. - Làm video demo các giải thuật.
Đinh Đức Tài	23122013	- Code, trực quan hóa mô hình bài toán tối ưu hóa rời rạc: TSP. - Code giải thuật trí tuệ bầy đàn ACO, PSO. - Code thuật toán truyền thống: GA. - Viết báo cáo TSP, ACO, PSO, GA. - Thực nghiệm, so sánh giải thuật cho TSP: ACO, PSO, GA, A*.

Các công nghệ sử dụng:

- **Python**: Python là một ngôn ngữ lập trình bậc cao, đa năng. Python được sử dụng rộng rãi trong các lĩnh vực như phát triển web, khoa học dữ liệu, trí tuệ nhân tạo, và tự động hóa.
- **Numpy**: NumPy là một thư viện Python mã nguồn mở, miễn phí, được sử dụng rộng rãi cho tính toán khoa học, khoa học dữ liệu và máy học. Nó cung cấp hỗ trợ hiệu quả cho các mảng đa chiều và các hàm toán học cấp cao.
- **Matplotlib**: Matplotlib là một thư viện lập trình mã nguồn mở phổ biến trong ngôn ngữ Python, được dùng để tạo ra các loại biểu đồ và hình ảnh chất lượng cao từ dữ liệu.

2 Các bài toán tối ưu hóa

2.1 Các benchmark tối ưu hóa liên tục

Để đánh giá và so sánh hiệu suất của các thuật toán tối ưu, nhóm đã triển khai ba hàm benchmark tiêu chuẩn cho bài toán tối ưu hóa liên tục gồm hàm **Sphere** và hàm **Rastrigin**. Tất cả các hàm đều được định nghĩa là bài toán tìm cực tiểu trong không gian D chiều.

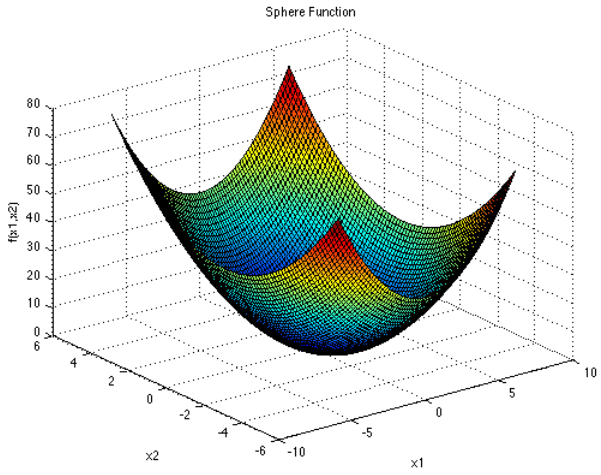
2.1.1 Hàm Sphere

Hàm Sphere (Hình 1) là một hàm lồi và đơn mô hình (unimodal). Đây là một trong những hàm benchmark đơn giản nhất, thường được sử dụng để kiểm tra tốc độ hội tụ cơ bản của một thuật toán. Công thức của hàm Sphere D chiều được định nghĩa là:

$$f(\mathbf{x}) = \sum_{i=1}^{D} x_i^2 \tag{1}$$

trong đó $\mathbf{x} = [x_1, x_2, ..., x_D]$ là vector đầu vào.

- **Cực tiểu toàn cục:** $f(\mathbf{x}^*) = 0$, đạt được tại $\mathbf{x}^* = (0, 0, ..., 0)$.
- **Phạm vi được chọn:** $-10 \leq x_i \leq 10$



Hình 1: Hàm Sphere ($D = 2$)

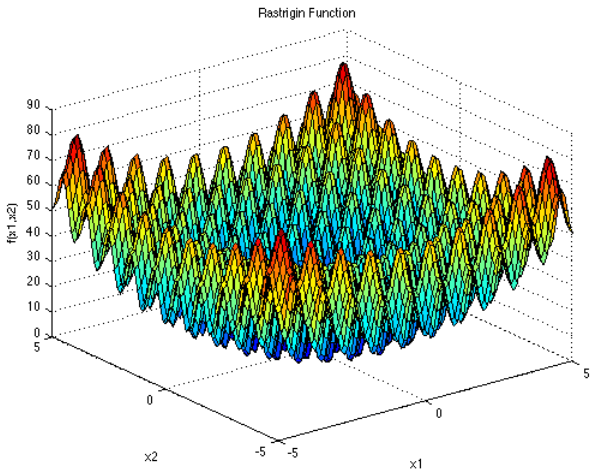
2.1.2 Hàm Rastrigin

Hàm Rastrigin (Hình 2) là một hàm đa mô hình (multimodal) phức tạp, đại diện cho một bề mặt gồ ghề với số lượng cực tiểu cục bộ rất lớn. Nó được tạo ra bằng cách thêm thành phần cosine vào hàm Sphere, khiến việc tìm kiếm cực tiểu toàn cục trở nên rất khó khăn. Công thức của hàm

Rastrigin D chiều được định nghĩa là:

$$f(\mathbf{x}) = 10D + \sum_{i=1}^{D}[x_i^2 - 10\cos(2\pi x_i)] \tag{2}$$

- **Cực tiểu toàn cục:** $f(\mathbf{x}^*) = 0$, đạt được tại $\mathbf{x}^* = (0, 0, ..., 0)$.
- **Phạm vi được chọn:** $-5.12 \leq x_i \leq 5.12$



Hình 2: Hàm Rastrigin ($D = 2$)

2.2 Bài toán tối ưu hóa rời rạc

2.2.1 Travelling salesman problem - TSP

Bài toán người bán hàng du lịch (*Travelling salesman problem - TSP* [3]) là một trong những bài toán tối ưu hóa tổ hợp được nghiên cứu rộng rãi nhất. Bài toán được phát biểu như sau: "Với một danh sách các thành phố và khoảng cách giữa mỗi cặp thành phố, đâu là tuyến đường ngắn nhất có thể đi qua mỗi thành phố đúng một lần và quay trở lại thành phố ban đầu?".

TSP còn được phát biểu dưới dạng toán học [4]:

- Cho $G = (V, E)$ là đồ thị, với $V = \{1, 2, ..., n\}$ là tập hợp gồm n đỉnh, E là tập hợp các cung hoặc cạnh. $C = (c_{ij})$ là ma trận khoảng cách (hoặc chi phí) ứng với E. Trong đó, c_{ij} là chi phí để đi trực tiếp từ đỉnh i đến đỉnh j.
- Với mỗi cặp đỉnh (i, j), $\forall i, j \in V$, đặt: $x_{ij} = \begin{cases} 1 & \text{nếu lộ trình có đi trực tiếp từ } i \text{ đến } j \\ 0 & \text{ngược lại} \end{cases}$
- Mục tiêu: Tối thiểu hóa hàm

$$Z = \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} \cdot x_{ij} \tag{3}$$

- Với các ràng buộc để đảm bảo chu trình Hamilton:

$$\sum_{j \in V, j \neq i} x_{ij} = 1, \forall i \in V; \sum_{i \in V, i \neq j} x_{ij} = 1, \forall j \in V \tag{4}$$

$$x_{ij} \in \{0, 1\}, \forall i, j \in V, i \neq j \tag{5}$$

Đặt u_i là thứ tự của đỉnh i trong lộ trình, $u_1 = 0; 1 \leq u_i \leq n-1, \forall i \in V \setminus \{1\}$:

$$\forall i, j \in V \setminus \{1\}, i \neq j : u_i - u_j + (n-1) \cdot x_{ij} \leq n-2 \tag{6}$$

Nói ngắn gọn, mục tiêu của *TSP* là **tìm chu trình Hamilton với độ dài ngắn nhất trên một đồ thị đầy đủ**. *TSP* là một bài toán NP-khó. Trong bài toán *TSP* đối xứng, khoảng cách giữa hai thành phố là không đổi dù đi theo chiều nào. Như vậy đồ thị trong bài toán này là đồ thị vô hướng. Ngược lại, với bài toán *TSP* bất đối xứng thì đường đi giữa hai thành phố có thể chỉ một chiều hoặc có độ dài khác nhau giữa mỗi chiều, tạo nên đồ thị có hướng.

3 Trí tuệ bầy đàn cho bài toán tối ưu hóa liên tục

3.1 Cuckoo Search

3.1.1 Mô tả thuật toán

Thuật toán *Cuckoo Search - CS* [5] được lấy cảm hứng từ chiến lược sinh sản ký sinh tổ của loài chim cuckoo. Các quy tắc lý tưởng hóa của thuật toán bao gồm:

- Mỗi con cuckoo đẻ một quả trứng (đại diện cho một giải pháp) tại một thời điểm và thả nó vào một cái tổ được chọn ngẫu nhiên.
- Những tổ tốt nhất sẽ được giữ lại và chuyển sang thế hệ tiếp theo.
- Số lượng tổ có sẵn là cố định. Chủ của tổ chim có thể phát hiện ra trứng lạ với xác suất $p_a \in [0, 1]$.
- Nếu bị phát hiện, chim chủ có thể vứt quả trứng đó đi hoặc từ bỏ tổ và xây một tổ mới hoàn toàn.

Đặc điểm cốt lõi của *Cuckoo Search* là việc sử dụng *Lévy flights* để tạo ra các giải pháp mới. *Lévy flights* là một kiểu nhảy ngẫu nhiên (random walk) có đặc điểm là các bước nhảy ngắn thường xuyên và thỉnh thoảng có các bước nhảy rất dài, tuân theo phân phối Lévy. Điều này giúp thuật toán thoát khỏi các điểm tối ưu cục bộ và khám phá không gian tìm kiếm hiệu quả hơn so với random walk tiêu chuẩn.

3.1.2 Tính toán Lévy FLights

Giải pháp mới dựa vào bước nhảy *Lévy* sẽ được tạo ra nhờ vào công thức sau:

$$x_i^{(t+1)} = x_i^{(t)} + \alpha \oplus Lévy(\beta) \tag{7}$$

Trong đó:

- $\alpha > 0$: Là hệ số dùng để điều chỉnh độ lớn bước nhảy.
- $L\acute{e}vy(\beta)$: Một vector ngẫu nhiên được rút ra từ phân phối *Lévy*.

Vì phân phối *Lévy* ở dạng tổng quát không có biểu thức giải tích đơn giản, bước nhảy *Lévy* sẽ được tạo ra bằng thuật toán *Mantegna* cho các phân phối ổn định. Các bước tính toán như sau:

1. Chọn một chỉ số β, với $1 < \beta \leq 2$ (mặc định là 1.5).
2. Tính toán phương sai (σ):
   - σ_v thường được đặt bằng 1.
   - σ_u được tính bằng công thức:

$$\sigma_u = \left(\frac{\Gamma(1+\beta)\sin(\pi\beta/2)}{\Gamma((1+\beta)/2)\beta 2^{(\beta-1)/2}} \right)^{1/\beta} \tag{8}$$

   (Trong đó $\Gamma(z)$ là hàm Gamma).
3. Tạo hai biến ngẫu nhiên u và v được rút ra từ phân phối chuẩn:
   - $u \sim N(0, \sigma_u^2)$
   - $v \sim N(0, \sigma_v^2)$
4. Tính bước nhảy Lévy: $L\acute{e}vy(\beta) = \frac{u}{|v|^{1/\beta}}$

3.1.3 Cơ chế hoạt động

Các bước thực hiện của thuật toán *Cuckoo search*:

1. **Khởi tạo**: Tạo một quần thể ban đầu gồm N tổ (giải pháp) x_i một cách ngẫu nhiên.
2. **Vòng lặp chính:** Lặp lại các bước 3 - 5 cho đến khi đạt số thế hệ tối đa.
3. **Lévy Flights**:
   - Chọn ngẫu nhiên một tổ i (giải pháp x_i)
   - Tạo ra một giải pháp mới x_{new} bằng cách áp dụng *Lévy Flight* lên x_i.
   - Đánh giá chất lượng F_{new} (fitness) của x_{new} (ở đây, F là hàm benchmark).
   - Chọn ngẫu nhiên một tổ khác j (giải pháp x_j).
   - Nếu F_{new} tốt hơn F_j (fitness của x_j), thì thay thế x_j bằng x_{new}.
4. **Loại bỏ tổ:** Loại bỏ $p_a \times N$ tổ có chất lượng kém nhất trong quần thể hiện tại và tạo ra các tổ ngẫu nhiên để thay thế.
5. **Đánh giá và cập nhật:** Đánh giá lại chất lượng của các tổ mới sau đó cập nhật lại giải pháp tốt nhất.

3.1.4 Mã giả

Algorithm 1 Cuckoo Search (CS)

// Thiết lập các giá trị N, max_iter, và p_a
Khởi tạo n tổ (giải pháp) ngẫu nhiên x_i $(i = 1, \ldots, N)$
Đánh giá fitness $F(x_i)$ cho tất cả các tổ
Ghi nhớ giải pháp tốt nhất G_{best}

while $t < max_iter$ **do**

  // Pha 1: Lévy Flights
  Chọn ngẫu nhiên một tổ i
  Tạo giải pháp mới $x_{\text{new}} \leftarrow x_i + \alpha \oplus \text{Lévy}(\lambda)$
  Chọn ngẫu nhiên một tổ j
  if $F(x_{\text{new}})$ tốt hơn $F(x_j)$ **then**
    $x_j \leftarrow x_{\text{new}}$

  // Pha 2: Loại bỏ tổ
  Sắp xếp các tổ và tìm ra $p_a \times N$ tổ tệ nhất
  for mỗi tổ tệ nhất k **do**
    Tạo giải pháp mới ngẫu nhiên x_{new}
    $x_k \leftarrow x_{\text{new}}$

  Cập nhật giải pháp tốt nhất G_{best}
  $t \leftarrow t + 1$
return G_{best}

3.2 Artificial Bee Colony

3.2.1 Mô tả thuật toán

Thuật toán *Artificial Bee Colony - ABC* [6] là một thuật toán tối ưu hóa lấy cảm hứng từ hành vi tìm kiếm thức ăn thông minh của bầy ong mật. Trong mô hình *ABC*, vị trí của một nguồn thức ăn đại diện cho một giải pháp của bài toán tối ưu, và lượng mật hoa của nguồn thức ăn đó tương ứng với "chất lượng" của giải pháp (ở đây là hàm benchmark).

Bầy ong được chia thành ba nhóm:

- **Ong thợ (Employed Bees):** Những con ong này được liên kết với một nguồn thức ăn cụ thể mà chúng đang khai thác. Chúng lưu trữ thông tin về nguồn thức ăn đó và chia sẻ nó với các con ong khác trong tổ.
- **Ong quan sát(Onlooker Bees):** Những con ong này chờ đợi trong tổ và quan sát các "điệu nhảy lắc lư" (waggle dances) của ong thợ. Chúng sẽ chọn một nguồn thức ăn dựa trên thông tin được chia sẻ, với xác suất lựa chọn tỷ lệ thuận với "chất lượng" của nguồn thức ăn đó.

- **Ong trinh sát (Scout Bees):** Đây là những con ong thợ đi khảo sát những nguồn thức ăn bị "cạn kiệt" (giải pháp không cải thiện sau một số lần thử nhất định). Chúng sẽ từ bỏ những nguồn thức ăn đó và tìm kiếm ngẫu nhiên nguồn thức ăn mới.

Thuật toán *ABC* duy trì sự cân bằng giữa hai quá trình khai thác (exploitation) và khám phá (exploration) nhờ vào cách hoạt động của ba nhóm ong này.

3.2.2 Cơ chế hoạt động

Với quần thể gồm N cá thể ong, ta tiến hành chia đều thành hai nhóm: nhóm *ong thợ* và nhóm *ong quan sát*. Trong đó, số lượng *ong thợ* đồng thời cũng là số nguồn thức ăn, được xác định bằng $\lfloor N/2 \rfloor$. Số lượng ong còn lại thuộc nhóm *ong quan sát*. Ở bước khởi tạo, ta tạo ngẫu nhiên $\lfloor N/2 \rfloor$ nguồn thức ăn trong không gian tìm kiếm và gán bộ đếm của mỗi nguồn thức ăn bằng 0 (được sử dụng trong pha *ong trinh sát*). Sau đó, thuật toán sẽ lặp lại tuần tự ba pha: *ong thợ*, *ong quan sát* và *ong trinh sát*, cho đến khi đạt tới số thế hệ tối đa.

Pha ong thợ (Employed Bee Phase): Trong pha này, mỗi ong thợ thực hiện tìm kiếm cục bộ xung quanh nguồn thức ăn (giải pháp x_i) mà nó đang quản lý. Mỗi ong thợ i tạo ra một ứng cử viên lân cận v_i với giải pháp x_i bằng cách sửa đổi chiều thứ j (chọn ngẫu nhiên) của giải pháp hiện tại và giữ nguyên các chiều còn lại. Chiều thứ j được sửa đổi bằng công thức:

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj}) \tag{9}$$

Trong đó:

- v_{ij} là giá trị chiều thứ j của giải pháp ứng viên v_i.
- x_{ij} là giá trị chiều thứ j của giải pháp hiện tại x_i.
- x_k là một giải pháp (nguồn thức ăn) khác được chọn ngẫu nhiên từ quần thể (với $k \neq i$).
- ϕ_{ij} là một số ngẫu nhiên trong khoảng $[-1, 1]$.

Sau khi tạo ra v_i, nếu "chất lượng" của giải pháp mới này tốt hơn thì thực hiện thay thế x_i thành v_i và cài lại bộ đếm i. Ngược lại, tăng bộ đếm i (đánh dấu giải pháp x_i).

Pha ong quan sát (Onlooker Bee Phase): Sau khi *ong thợ* đã hoàn thành tìm kiếm, chúng chia sẻ thông tin về chất lượng nguồn thức ăn. Mỗi *ong quan sát* sẽ khai thác nguồn thức ăn như sau:

1. **Tính xác suất lựa chọn:** Xác suất p_i của nguồn x_i được tính bằng công thức:

$$p_i = \frac{Nectar(fit_i)}{\sum_{n=1}^{N} Nectar(fit_n)} \tag{10}$$

Trong đó:

- N: Số nguồn thức ăn.

- $Nectar(fit_i)$: Giá trị lợi nhuận được chuẩn hóa từ fitness. Đối với bài toán *tối thiểu hóa* thì được tính bằng công thức:

$$Nectar(fit_i) = \begin{cases} \dfrac{1}{1+fit_i}, & \text{nếu } fit_i \geq 0, \\ 1+|fit_i|, & \text{nếu } fit_i < 0. \end{cases} \tag{11}$$

2. **Lựa chọn nguồn:** *Ong quan sát* sẽ chọn một nguồn thức ăn i bằng phương pháp chọn Roulette Wheel dựa trên xác suất p_i
3. **Khai thác:** Khi đã chọn được nguồn khai thác, ong quan sát sẽ thực hiện khai thác giống như ong thợ (tạo ứng cử viên v_i, so sánh, cập nhật).

Bằng cách khai thác nguồn thức ăn như trên, thuật toán sẽ tập trung nhiều vào các giải pháp có vẻ hứa hẹn hơn (có chất lượng cao hơn).

Pha ong trinh sát (Scout Bee Phase): Ong trinh sát sẽ kiểm tra bộ đếm của mỗi giải pháp (nguồn thức ăn) nếu một giải pháp không được cải thiện sau số lượng chu kỳ nhất định (vượt *limit*) thì nguồn thức ăn đó được coi là "cạn kiệt". Lúc này, ong trinh sát sẽ trở thành ong thợ và thực hiện "từ bỏ" nguồn thức ăn cũ để tìm kiếm nguồn thức ăn mới bằng cách tạo ra một giải pháp mới ngẫu nhiên trong không gian tìm kiếm.

3.3 Firefly Algorithm

3.3.1 Mô tả thuật toán

Firefly Algorithm - FA [7] là một thuật toán lấy cảm hứng từ hành vi nhấp nháy (phát quang sinh học) và sự thu hút lẫn nhau của các loài đom đóm. Ý tưởng cốt lõi là mỗi đom đóm (đại diện cho một giải pháp) có một "độ sáng" tỷ lệ thuận với "chất lượng" của giải pháp đó (ở đây là hàm benchmark). Các con đom đóm sẽ bị thu hút lẫn nhau dựa trên độ sáng này. Các quy tắc lý tưởng hóa của thuật toán bao gồm:

- Tất cả đom đóm là phi giới tính (unisex), tức là một con đom đóm có thể bị thu hút bởi một con đom đóm khác bất kể giới tính.
- Độ hấp dẫn tỷ lệ thuận với độ sáng. Do đó đom đóm ít sáng hơn sẽ di chuyển về phía đom đóm sáng hơn.
- Độ hấp dẫn và độ sáng đều giảm khi khoảng cách giữa chúng tăng lên, do ánh sáng bị hấp thụ bởi môi trường.
- Nếu không có con đom đóm nào sáng hơn một con đom đóm cụ thể, nó sẽ di chuyển ngẫu nhiên.

3.3.2 Tính toán độ hấp dẫn

Để mô hình hóa sự suy giảm độ hấp dẫn, thuật toán sử dụng một hệ số hấp thụ ánh sáng γ. Độ hấp dẫn β tại một khoảng cách r được định nghĩa như sau:

$$\beta(r) = \beta_0 e^{-\gamma r^2} \tag{12}$$

Trong đó β_0 là độ hấp dẫn tại khoảng cách $r = 0$. Khi con đom đóm i (ít sáng hơn) bị thu hút bởi con đom đóm j (sáng hơn), vị trí mới của i được xác định bởi công thức:

$$x_i^{(t+1)} = x_i^{(t)} + \beta(r_{ij}^2)(x_j^{(t)} - x_i^{(t)}) + \alpha\epsilon_i \tag{13}$$

Công thức bao gồm ba thành phần:

1. $x_i^{(t)}$: Vị trí của đom đóm i tại thời điểm t.
2. $\beta(r_{ij}^2)(x_j^{(t)} - x_i^{(t)})$ (Thành phần hấp dẫn):
   - Giúp đom đóm i di chuyển về phía đom đóm j theo độ hấp dẫn.
   - $\beta(r_{ij}^2)$ là độ hấp dẫn của đom đóm j đối với đom đóm i.
   - $(x_j^{(t)} - x_i^{(t)})$ là vector chỉ phương.
3. $\alpha\epsilon_i$ (Thành phần ngẫu nhiên):
   - Đảm bảo sự khám phá (exploration) cho đom đóm, tránh đi thẳng về phía giải pháp tốt nhất và bị kẹt sớm ở giải pháp cục bộ.
   - α là tham số điều khiển mức độ ngẫu nhiên.
   - ϵ_i là một vector gồm các số ngẫu nhiên được rút ra từ một phân phối (ví dụ như phân phối Lévy).

3.3.3 Cơ thế hoạt động

Các bước hoạt động của *Firefly Algorithm*:

1. **Khởi tạo:**
   - Tạo một quần thể ban đầu gồm N đom đóm (giải pháp) ngẫu nhiên.
   - Đánh giá độ sáng $F(i), \forall i \in \{1, 2, \ldots, N\}$. Vì đang áp dụng cho bài toán tối tiểu hóa nên ta ngầm hiểu rằng I_i càng nhỏ thì đom đóm đó càng sáng.
2. **Vòng lặp chính:** Thực hiện bước 3 và 4 đến khi đạt số thế hệ tối đa.
3. **Di chuyển đom đóm:** Với mỗi đom đóm i và j $(i \neq j)$, nếu đom đóm j sáng hơn i thì di chuyển x_i theo công thức 13 và cập nhật độ sáng I_i.
4. **Cập nhật giải pháp:** Cập nhật lại giải pháp tốt nhất sau khi di chuyển các đom đóm.

3.3.4 Mã giả

Algorithm 2 Firefly Algorithm (FA)

Khởi tạo quần thể gồm N đom đóm x_i $(i = 1, \ldots, N)$
Đánh giá độ sáng (fitness) $I_i = F(x_i), \forall i \in \{1, 2, \ldots, N\}$.
Thiết lập các tham số: β_0, γ, α

while $t < MaxGeneration$ **do**
  for $i = 1$ **to** n **do**
    for $j = 1$ **to** n **do**
      // Đom đóm j sáng hơn i (bài toán tối thiểu hóa)
      if $I_j < I_i$ **then**
        Tính khoảng cách $r_{ij} = ||x_i - x_j||$
        Tính độ hấp dẫn $\beta = \beta_0 e^{-\gamma r_{ij}^2}$
        Sinh bước ngẫu nhiên ϵ_i
        Cập nhật vị trí: $x_i \leftarrow x_i + \beta(x_j - x_i) + \alpha\epsilon_i$
        Đánh giá lại độ sáng
  Cập nhật giải pháp tốt nhất G_{best}
  $t \leftarrow t + 1$

return G_{best}

4 Trí tuệ bầy đàn cho bài toán tối ưu hóa rời rạc

4.1 Ant Colony Optimization

Giải thuật tối ưu đàn kiến (*Ant Colony Optimization - ACO* [8]) là một siêu heuristic (meta-heuristic) dựa trên xác suất, lấy cảm hứng từ hành vi tìm kiếm thức ăn của loài kiến trong tự nhiên. Ý tưởng của *ACO* là mô phỏng một tập hợp các "kiến nhân tạo" hợp tác với nhau để tìm ra lời giải tốt cho một bài toán. Sự hợp tác này được thực hiện một cách gián tiếp thông qua việc để lại các dấu vết "pheromone nhân tạo" trên các thành phần của lời giải, tương tự như cách loài kiến thật sử dụng cơ chế giao tiếp gián tiếp gọi là stigmergy (giao tiếp qua môi trường), thông qua việc để lại *pheromone* hóa học để đánh dấu đường đi.

4.1.1 ACO và tối ưu hóa tổ hợp

ACO hiệu quả trong việc giải quyết các bài toán tối ưu hóa tổ hợp (*combinatorial optimization problem*) và tìm đường đi trên đồ thị (như *TSP*). Mô hình $P = (\mathbf{S}, \Omega, f)$ của bài toán tối ưu hóa tổ hợp gồm các thành phần:

- Không gian tìm kiếm $\mathbf{S}$ xác định trên một tập hợp hữu hạn các biến quyết định rời rạc $X_i, i = 1, \ldots, n$.
- Tập hợp Ω gồm ràng buộc giữa các biến.
- Hàm mục tiêu $f : \mathbf{S} \to \mathbb{R}_0^+$ để tối thiểu hóa.

Biến X_i nhận giá trị trong tập hợp $\mathbf{D}_i = \{v_i^1, \ldots, v_i^{|\mathbf{D}_i|}\}$. Một giải pháp khả thi $s \in \mathbf{S}$ là một phép gán hoàn chỉnh các giá trị cho các biến thỏa mãn tất cả các ràng buộc trong Ω. Một nghiệm $s^* \in \mathbf{S}$ gọi là nghiệm tối ưu toàn cục khi và chỉ khi: $\forall s \in \mathbf{S} : f(s^*) \leq f(s)$

4.1.2 Cơ chế hoạt động ACO

Quá trình hoạt động của *ACO* là một vòng lặp gồm ba bước chính:

1. Xây dựng giải pháp của Kiến (Construct Ant Solutions):
   - Một đàn gồm m "kiến nhân tạo" sẽ xây dựng các giải pháp từ các phần tử của một tập hợp hữu hạn các thành phần giải pháp có sẵn $\mathbf{C} = \{c_{ij}\}, i = 1, \ldots, n; j = 1, \ldots, |\mathbf{D}_i|$.
   - Một giải pháp được xây dựng từ tập hợp $s^p = \emptyset$. Với mỗi bước xây dựng, s^p được mở rộng bằng cách thêm một phần tử từ tập hợp $\mathbf{N}(s^p) \subseteq \mathbf{C}$, với $\mathbf{N}(s^p)$ là tập hợp các thành phần có thể được thêm vào s^p mà không vi phạm ràng buộc trong Ω.
   - Tại mỗi bước, việc lựa chọn thành phần tiếp theo từ $\mathbf{N}(s^p)$ dựa vào cơ chế ngẫu nhiên, nhưng bị ảnh hưởng bởi:
     - Dấu vết Pheromone (τ) ứng với từng phần tử trong $\mathbf{N}(s^p)$.
     - Thông tin Heuristic (η).
   - Việc xây dựng các giải pháp có thể được so sánh với việc tạo đường đi / chu trình trên đồ thị $G_C = (V, E)$.
2. Tìm kiếm cục bộ: Trước khi cập nhật pheromone, các giải pháp được kiến đưa ra thường sẽ được tối ưu dựa trên tìm kiếm cục bộ. Bước này không bắt buộc nhưng thường có trong các thuật giải ACO hiện đại.
3. Cập nhật pheromone: Mục đích của việc cập nhật pheromone là tăng giá trị pheromone liên quan đến các giải pháp tốt hoặc có triển vọng, và giảm giá trị pheromone liên quan đến các giải pháp xấu.
   - Giảm tất cả các giá trị pheromone thông qua quá trình bay hơi pheromone (*pheromone evaporation*).
   - Tăng nồng độ pheromone ứng với tập hợp các giải pháp tốt.

4.1.3 Thuật toán ACO

ACO có nhiều phiên bản, gồm bản gốc *Ant System* và các biến thể như *MAX-MIN Ant System*, *Ant Colony System*. Với bản *Ant System*, đặc điểm chính của nó là ở mỗi vòng lặp, giá trị pheromone được cập nhật bởi tất cả m "kiến nhân tạo" mà đã xây dựng giải pháp trong vòng lặp đó.

Xét bài toán *TSP*, giá trị pheromone τ_{ij} ứng với cạnh nối giữa 2 thành phố i và j được cập nhật như sau:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^k \tag{14}$$

với ρ là tốc độ bay hơi pheromone, m là số lượng kiến, $\Delta\tau_{ij}^k$ là lượng pheromone để lại bởi con kiến k:

$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{nếu kiến } k \text{ sử dụng lộ trình có cạnh } (i,j) \\ 0 & \text{ngược lại} \end{cases} \tag{15}$$

với Q là hằng số và L_k là độ dài của hành trình đi bởi con kiến k.

Các con kiến xây dựng giải pháp dựa vào cơ chế ngẫu nhiên. Với con kiến k đang ở thành phố i và xây dựng giải pháp s^p, xác suất để đi đến thành phố j là:

$$p_{ij}^k = \begin{cases} \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{c_{il} \in \mathbf{N}(s^p)} \tau_{il}^\alpha \cdot \eta_{il}^\beta} & \text{nếu } c_{ij} \in \mathbf{N}(s^p) \\ 0 & \text{ngược lại} \end{cases} \tag{16}$$

với $\mathbf{N}(s^p)$ là tập hợp các thành phần khả thi: cạnh (i,l) với l là thành phố chưa được thăm bởi kiến k. α và β là trọng số của pheromone và thông tin heuristic η_{ij}:

$$\eta_{ij} = \frac{1}{d_{ij}} \tag{17}$$

với d_{ij} là khoảng cách giữa thành phố i và j.

4.2 Particle Swarm Optimization

Giải thuật tối ưu hóa bầy hạt (*Particle Swarm Optimization - PSO* [9]) là thuật toán tìm kiếm bằng quần thể, dựa trên việc mô phỏng hành vi xã hội của các đàn chim. *PSO* ban đầu được xây dựng cho các bài toán tối ưu liên tục, nhưng vẫn có phiên bản chỉnh sửa để phục vụ bài toán tối ưu rời rạc.

Trong *PSO*, các cá thể, hay còn được gọi là các *hạt* (*particles*), được "bay" qua một không gian tìm kiếm siêu chiều. Sự thay đổi vị trí của các *hạt* trong không gian tìm kiếm dựa trên xu hướng tâm lý xã hội của các cá thể trong việc bắt chước thành công của những cá thể khác. Sự thay đổi của một *hạt* trong bầy đàn bị ảnh hưởng bởi kinh nghiệm, kiến thức của các *hạt* lân cận. Vì vậy, hành vi tìm kiếm của một *hạt* bị ảnh hưởng bởi hành vi của các *hạt* khác trong bầy đàn (do đó, *PSO* là một loại thuật toán hợp tác cộng sinh). Việc mô phỏng hành vi xã hội này có hệ quả là quá trình tìm kiếm diễn ra theo cách các *hạt* ngẫu nhiên quay trở lại các vùng đã thành công trước đó trong không gian tìm kiếm.

4.2.1 Thuật toán PSO cơ bản

Thuật toán *PSO* duy trì một bầy các hạt, trong đó mỗi hạt đại diện cho một giải pháp tiềm năng. Tương tự như các mô hình tính toán tiến hóa, một bầy đàn (swarm) được xem như một quần thể (population), và một hạt (particle) được xem như một cá thể (individual). Nói một cách đơn giản, các hạt này "bay" trong một không gian tìm kiếm đa chiều, tại đó vị trí của mỗi hạt được điều chỉnh dựa trên kinh nghiệm của chính nó và của các hạt lân cận.

Đặt $\mathbf{x}_i(t)$ biểu diễn vị trí của hạt i trong không gian tìm kiếm ở thời gian t (t biểu diễn các thời gian rời rạc). Vị trí của hạt i bị thay đổi bằng cách thêm vận tốc $\mathbf{v}_i(t)$ vào vị trí hiện tại:

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \tag{18}$$

với $\mathbf{x}_i(0) \sim U(\mathbf{x}_{min}, \mathbf{x}_{max})$

Vector vận tốc $\mathbf{v}_i(t)$ này điều khiển quá trình tối ưu hóa của PSO. $\mathbf{v}_i(t)$ phản ánh cả kinh nghiệm của chính hạt i (*cognitive component*) và thông tin trao đổi xã hội từ các hạt lân cận (*social component*).

Ban đầu, hai thuật toán *PSO* được phát triển là *gbest PSO* và *lbest PSO*. Chúng khác nhau về định nghĩa phạm vi "lân cận" (neighborhood).

4.2.2 Global Best PSO - gbest PSO

Với *Global Best PSO* (*gbest PSO*), phạm vi lân cận của một hạt là toàn bộ bầy hạt. Tất cả các hạt trong bầy đàn đều bị ảnh hưởng và thu hút bởi một vị trí duy nhất: vị trí tốt nhất đã từng được tìm thấy bởi bất kỳ hạt nào trong toàn bộ bầy đàn từ trước đến nay (được gọi là *gbest* - $\widehat{\mathbf{y}}(t)$). Vận tốc của hạt i được tính bằng:

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\widehat{y}_j(t) - x_{ij}(t)] \tag{19}$$

với $v_{ij}(t)$ là vận tốc của hạt i tại chiều $j \in \{1, \ldots, n_x\}$ ở thời gian t; $x_{ij}(t)$ là vị trí của hạt i tại chiều j ở thời gian t; c_1 và c_2 là trọng số của thành phần kinh nghiệm cá nhân (*cognitive*) và kinh nghiệm bầy hạt (*social*); $r_{1j}, r_{2j} \sim U(0,1)$ là giá trị ngẫu nhiên lấy từ phân phối đều liên tục, trong khoảng $[0,1]$ (thành phần ngẫu nhiên trong thuật toán *PSO*).

Vị trí tốt nhất $\mathbf{y}_i$ của hạt i là vị trí tốt nhất mà hạt i tìm được kể từ thời gian t đầu tiên:

$$\mathbf{y}_i(t+1) = \begin{cases} \mathbf{y}_i(t) & \text{nếu } f(\mathbf{x}_i(t+1)) \geq f(\mathbf{y}_i(t)) \\ \mathbf{x}_i(t+1) & \text{nếu } f(\mathbf{x}_i(t+1)) < f(\mathbf{y}_i(t)) \end{cases} \tag{20}$$

với $f : \mathbb{R}^{n_x} \to \mathbb{R}$ là hàm thích nghi. Hàm này đánh giá chất lượng hoặc hiệu suất của một hạt (một lời giải).

Vị trí tốt nhất toàn cục $\widehat{\mathbf{y}}(t)$ ở thời gian t là vị trí tốt nhất trong kinh nghiệm của bầy hạt:

$$\widehat{\mathbf{y}}(t) \in \{\mathbf{y}_0(t), \ldots, \mathbf{y}_{n_s}(t)\} | f(\widehat{\mathbf{y}}(t)) = min\{f(\mathbf{y}_0(t)), \ldots, f(\mathbf{y}_{n_s}(t))\} \tag{21}$$

với n_s là tổng số hạt trong bầy hạt.

Vị trí tốt nhất toàn cục $\widehat{\mathbf{y}}(t)$ ở thời gian t còn có thể được tính bằng vị trí tốt nhất của các hạt ở thời gian t hiện tại:

$$\widehat{\mathbf{y}}(t) = min\{f(\mathbf{x}_0(t)), \ldots, f(\mathbf{x}_{n_s}(t))\} \tag{22}$$

Một phiên bản cải tiến của PSO [10] đã thêm trọng số quán tính w, giúp cân bằng giữa khả năng khám phá (exploration) và khả năng khai thác (exploitation) không gian tìm kiếm:

$$v_{ij}(t+1) = wv_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\widehat{y}_j(t) - x_{ij}(t)] \tag{23}$$

với w lớn ($w \sim 1$), hạt có xu hướng giữ lại nhiều vận tốc cũ của nó, giúp nó di chuyển xa hơn và "khám phá" các vùng mới trong không gian tìm kiếm; với w nhỏ ($w \sim 0$), hạt bị ảnh hưởng nhiều hơn (*pbest*) và (*gbest*), giúp nó hội tụ và "khai thác" kỹ hơn xung quanh các giải pháp tốt đã tìm thấy.

4.2.3 Local Best PSO - lbest PSO

Với *Local Best PSO* (*lbest PSO*), phạm vi lân cận của một hạt nhỏ hơn *gbest PSO*, không phải toàn bộ bầy hạt. Lân cận của *lbest PSO* có thể định nghĩa bằng khoảng cách hoặc đánh trước chỉ số cho hạt. Vận tốc hạt i được tính bằng:

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\widehat{y}_{ij}(t) - x_{ij}(t)] \tag{24}$$

với $\widehat{y}_{ij}$ là vị trí tốt nhất được tìm thấy bởi lân cận của hạt i trong chiều j.

Vị trí tốt nhất địa phương $\widehat{\mathbf{y}}_i$ là vị trí tốt nhất được tìm thấy trong lân cận $\mathcal{N}_i$:

$$\widehat{\mathbf{y}}_i(t+1) \in \{\mathcal{N}_i | f(\widehat{\mathbf{y}}_i(t+1)) = min\{f(x)\}, \forall x \in \mathcal{N}_i\} \tag{25}$$

với định nghĩa của lân cận là:

$$\mathcal{N}_i = \{\mathbf{y}_{i-n_{\mathcal{N}_i}}(t), \mathbf{y}_{i-n_{\mathcal{N}_i}+1}(t), \ldots, \mathbf{y}_{i-1}(t), \mathbf{y}_i(t), \mathbf{y}_{i+1}(t), \ldots, \mathbf{y}_{i+n_{\mathcal{N}_i}}(t)\} \tag{26}$$

với kích thước của lân cận là $\mathcal{N}_i$. Vị trí tốt nhất địa phương cũng chính là vị trí tốt nhất của lân cận.

Tương tự, ta có phiên bản thêm trọng số quán tính w cho công thức tính vận tốc hạt i:

$$v_{ij}(t+1) = wv_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\widehat{y}_{ij}(t) - x_{ij}(t)] \tag{27}$$

4.2.4 Điều chỉnh thuật toán PSO cho tối ưu hóa rời rạc

Thuật toán *PSO* dùng tốt nhất cho bài toán tối ưu hóa liên tục. Vì vậy, ta cần định nghĩa lại các *hạt*, vận tốc của *PSO* để phù hợp với bài toán tối ưu hóa rời rạc. Với bài toán *TSP*, phương pháp đơn giản là xem một *hạt* là một hoán vị của các thành phố [11] (một chu trình).

Xét bài toán *TSP* đối xứng có m thành phố, đặt $X_i = \{x_{i1}, x_{i2}, \ldots, x_{im}\}$ là vị trí của hạt i trong bầy *PSO*, thể hiện đường đi $x_{i1} \to x_{i2} \to \cdots \to x_{im} \to x_{i1}$. Ta có các định nghĩa sau với $X = \{x_1, x_2, \ldots, x_m\}$, $A = \{a_1, a_2, \ldots, a_m\}$:

- Phép dịch vòng: $SL(X, k) = \{x_{(1+k)\%m}, x_{(2+k)\%m}, \ldots, x_{(m+k)\%m}\}$
- Phép đảo ngược: $RE(X) = \{x_m, x_{m-1}, \ldots, x_1\}$
- Với *hạt* X: $X = SL(X, k) = RE(X)$ do chi phí di chuyển là bằng nhau.

- Một phép chuyển vị T giữa 2 phần tử trong tập A là một hoán vị f sao cho: $f(a_i) = a_j$, $f(a_j) = f(a_i)$, $f(a_g) = a_g$ $(\forall g \in \{1, 2, \ldots, m\} \setminus \{i, j\})$.
- $P(A, B) = T_1 \cdot T_2 \cdots T_k$ là chuỗi chuyển vị độ dài k để biến A thành B. Với k là độ dài chuỗi chuyển vị bé nhất, ta định nghĩa $B - A = T_1 \cdot T_2 \cdots T_k$.

Đến đây, ta có thể định nghĩa các phép toán cho *PSO* để giải bài toán *TSP*. Với hai *hạt* $X_i = \{x_{i1}, x_{i2}, \ldots, x_{im}\}$, $X_j = \{x_{j1}, x_{j2}, \ldots, x_{jm}\}$, vận tốc $V_1 = T_1T_2 \ldots T_k$, $V_2 = T_{k+1}T_{k+2} \ldots T_l$ và $r \in \mathbb{R}$:

- Phép trừ vị trí: $X_j - X_i = V = \min_k \{T_1T_2 \ldots T_k | \exists l : P(X_i, SL(X_j, l)) = T_1T_2 \ldots T_k \vee P(X_i, SL(RE(X_j, l)) = T_1T_2 \ldots T_k\}$ [1]
- Phép cộng vận tốc: $V_1 + V_2 = T_1T_2 \ldots T_k + T_{k+1}T_{k+2} \ldots T_l = T_1T_2 \ldots T_k \ldots T_l$
- Phép nhân vô hướng với vận tốc: $V_1 \cdot r = T_1T_2 \ldots T_k \cdot r = T_1T_2 \ldots T_{\lfloor r \cdot |V_1| \rfloor}$, $0 \leq r \leq 1$
- Phép cộng vị trí và vận tốc: $X_1 + V = T_k(\ldots T_2(T_1(X_1)) \ldots)$

5 Thuật toán truyền thống cho các bài toán tối ưu

5.1 Hill Climbing cho bài toán tối ưu hóa liên tục

5.1.1 Mô tả thuật toán

Giải thuật leo đồi *(Hill Climbing)* là một kỹ thuật tối ưu hóa cục bộ dựa trên nguyên tắc lặp đi lặp lại để tìm ra một giải pháp tốt hơn. Đây là một trong những thuật toán tìm kiếm cục bộ tham lam đơn giản nhất:

- Bắt đầu với một giải pháp bất kỳ, sau đó sử dụng một vòng lặp liên tục di chuyển theo hướng tối ưu, cố gắng tìm một giải pháp tốt hơn.
- Dừng lại khi đạt đến điểm tối ưu cục bộ (local optima) mà không có hàng xóm nào có giá trị tối ưu hơn.

Thuật toán chỉ xem xét các giải pháp lân cận trực tiếp của trạng thái hiện tại, vì vậy nó có thể dễ dàng bị mắc kẹt trong các điểm tối ưu cục bộ – những giải pháp tốt hơn các giải pháp lân cận nhưng không phải là giải pháp tốt nhất trong toàn bộ không gian tìm kiếm. Và đây cũng chính là nhược điểm lớn nhất của Hill Climbing.

5.1.2 Thành phần cốt lõi

Cốt lõi của Hill Climbing nằm ở hai yếu tố: Cách tạo ra giải pháp lân cận và quy tắc chấp nhận giải pháp đó.

[1] Trong công thức gốc [11], tác giả định nghĩa là $SL(P(X_i, X_j), l))$, $SL(RE(P(X_i, X_j), l)))$. Đây là định nghĩa không tường minh vì phép $SL(X, l)$, $RE(X)$ cần được áp dụng cho một hoán vị trước khi tìm chuỗi chuyển vị.

Tạo giải pháp lân cận: Tại mỗi bước, một giải pháp mới s_{new} được tạo ra bằng cách áp dụng một phép biến đổi nhỏ và ngẫu nhiên lên giải pháp hiện tại $s_{current}$. Phép biến đổi này phụ thuộc vào không gian tìm kiếm của bài toán. Trong không gian liên tục, có thể tạo ra các giải pháp lân cận bằng cách thêm một vector nhiễu nhỏ vào vector tham số hiện tại:

$$s_{new} = s_{current} + \texttt{random_vector}(-\texttt{step_size}, \texttt{step_size}) \tag{28}$$

Trong đó: Giá trị `step_size` xác định khoảng cách của vùng lân cận được khám phá.

Quy tắc chấp nhận của Hill Climbing được xác định:

- Tính toán chi phí của giải pháp mới: $\text{cost}(s_{new})$.
- So sánh $\text{cost}(s_{new})$ với chi phí của giải pháp hiện tại $\text{cost}(s_{current})$.
- Nếu $\text{cost}(s_{new}) < \text{cost}(s_{current})$, giải pháp mới sẽ được chấp nhận và trở thành giải pháp hiện tại cho vòng lặp tiếp theo.

Khác với Simulated Annealing, Hill Climbing không bao giờ chấp nhận một giải pháp tệ hơn.

5.1.3 Cơ chế hoạt động

Các bước thực hiện của thuật toán *Hill Climbing*:

1. **Khởi tạo**:
   - Tạo một giải pháp ban đầu ngẫu nhiên $s_{current}$.
   - Ghi nhận là giải pháp tốt nhất đã tìm thấy, $s_{best} = s_{current}$.
2. **Vòng lặp chính**: Lặp lại các bước sau có giới hạn số lần lặp tối đa.
   (a) **Tạo giải pháp lân cận**: Tạo một giải pháp lân cận s_{new} từ $s_{current}$.
   (b) **Đánh giá và so sánh**:
      - Nếu $\text{cost}(s_{new}) < \text{cost}(s_{current})$:
        – Chấp nhận giải pháp mới: $s_{current} = s_{new}$.
   (c) **Cập nhật giải pháp tốt nhất**: Nếu $\text{cost}(s_{current}) < \text{cost}(s_{best})$, cập nhật: $s_{best} = s_{current}$.
3. **Kết thúc**: Trả về giải pháp tốt nhất đã tìm được, s_{best}.

5.2 Simulated Annealing cho bài toán tối ưu hóa liên tục

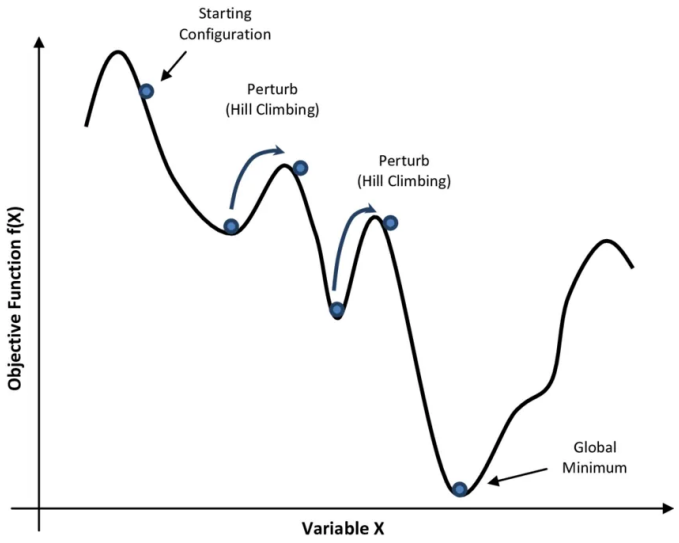
5.2.1 Mô tả thuật toán

Giải thuật mô phỏng luyện kim (*Simulated Annealing - SA* [12]) là một kỹ thuật tối ưu hóa xác suất, lấy cảm hứng từ quá trình luyện kim trong vật lý. Giải thuật có thể được hình dung như khi ta thả một quả bóng xuống một bề mặt mấp mô:

- Nếu ta để quả bóng tự lăn thì nó sẽ dừng ở cực tiểu cục bộ.

- Nếu ta lắc bề mặt thì ta có thể tung quả bóng ra khỏi cực tiểu cục bộ.
- Giải pháp mô phỏng luyện kim là bắt đầu việc lắc mạnh (ứng với nhiệt độ cao), thuật toán có xu hướng chấp nhận cả những giải pháp kém hơn, cho phép quả bóng nhảy ra khỏi cực tiểu cục bộ.
- Sau đó từ từ giảm cường độ lắc (giảm dần nhiệt độ) để lực không quá mạnh đánh bật nó ra khỏi cực tiểu toàn cục, xác suất chấp nhận giải pháp kém hơn giảm theo, giúp thuật toán hội tụ về một giải pháp tốt

Giải thuật mô phỏng luyện kim là sự kết hợp của hai kỹ thuật *Hill Climbing (leo đồi)* và `Pure Random Walk (bước đi ngẫu nhiên)`. Kỹ thuật `Hill Climbing` giúp tìm giá trị cực trị toàn cục, còn kỹ thuật `Pure Random Walk` giúp tăng hiệu quả tìm kiếm giá trị tối ưu.



Hình 3: Mô phỏng giải thuật Simulated Annealing (SA)

5.2.2 Thành phần cốt lõi

Tương tự như Hill Climbing, cốt lõi của Simulated Annealing nằm ở hai yếu tố: Cách tạo ra giải pháp lân cận và quy tắc chấp nhận giải pháp đó.

Tạo giải pháp lân cận: Tại mỗi bước, một giải pháp mới s_{new} được tạo ra bằng cách áp dụng một phép biến đổi nhỏ và ngẫu nhiên lên giải pháp hiện tại $s_{current}$. Phép biến đổi này phụ thuộc vào không gian tìm kiếm của bài toán. Trong không gian liên tục, có thể tạo ra các giải pháp lân cận bằng cách thêm một vector nhiễu nhỏ vào vector tham số hiện tại:

$$s_{new} = s_{current} + \texttt{random_vector}(-\texttt{step_size}, \texttt{step_size}) \tag{29}$$

Trong đó: Giá trị `step_size` xác định khoảng cách của vùng lân cận được khám phá.

Hàm xác suất chấp nhận: Quyết định chấp nhận giải pháp mới s_{new} được đưa ra dựa trên tiêu chuẩn *Metropolis-Hastings*.

- Đầu tiên, ta tính sự thay đổi về giá trị của hàm mục tiêu $\Delta E = \text{cost}(s_{new}) - \text{cost}(s_{current})$.
- Nếu $\Delta E < 0$ (tức là s_{new} tốt hơn $s_{current}$), giải pháp mới luôn được chấp nhận.
- Nếu $\Delta E \geq 0$ (tức là s_{new} không tốt bằng hoặc tệ hơn $s_{current}$), giải pháp vẫn có thể được chấp nhận với một xác suất P phụ thuộc vào nhiệt độ hiện tại T:

$$P(\Delta E, T) = \exp\left(-\frac{\Delta E}{T}\right) \tag{30}$$

  Việc chấp nhận giải pháp tệ hơn giúp thuật toán tránh bị mắc kẹt trong các điểm tối ưu cục bộ.

5.2.3 Cơ chế hoạt động

Các bước thực hiện của giải thuật *Simulated Annealing*:

1. **Khởi tạo**:
   - Tạo một giải pháp ban đầu ngẫu nhiên $s_{current}$.
   - Đặt nhiệt độ ban đầu T ở mức cao (T_{max}).
   - Gán giá trị $s_{best} = s_{current}$.
   - Khai báo hệ số làm nguội: α, $(0 < \alpha < 1)$.
2. **Vòng lặp chính**: Lặp lại các bước sau cho đến khi nhiệt độ T đạt đến ngưỡng tối thiểu (T_{min}) hoặc thỏa mãn điều kiện dừng.
   (a) **Vòng lặp phụ (tại nhiệt độ T)**: Lặp lại L lần nhất định.
      - Tạo một giải pháp lân cận s_{new} từ $s_{current}$.
      - Tính $\Delta E = \text{cost}(s_{new}) - \text{cost}(s_{current})$.
      - Nếu $\Delta E < 0$ hoặc $\text{random}(0, 1) < \exp(-\Delta E/T)$:
        – Chấp nhận giải pháp mới: $s_{current} = s_{new}$.
      - Nếu $\text{cost}(s_{current}) < \text{cost}(s_{best})$, cập nhật giải pháp tốt nhất: $s_{best} = s_{current}$.

   (b) **Làm nguội**: Giảm nhiệt độ $T = T \times \alpha$.
3. **Kết thúc**: Trả về giải pháp tốt nhất đã tìm được s_{best}.

5.3 Tìm kiếm A* cho bài toán tối ưu hóa rời rạc

5.3.1 Mô tả thuật toán

Thuật toán A* [13] là một thuật toán tìm đường đi ngắn nhất giữa hai điểm trong không gian đồ thị. Đây là một thuật toán tìm kiếm có thông tin (*informed search*), sử dụng một hàm heuristic để ước tính chi phí đến đích, giúp dẫn hướng quá trình tìm kiếm một cách hiệu quả hơn so với các thuật toán tìm kiếm mù (uninformed search) như BFS hay Dijkstra.

Thuật toán A* nổi bật nhờ sự kết hợp những ưu điểm của hai thuật toán:

- **Thuật toán Dijkstra:** Thuật toán này tìm đường đi ngắn nhất đến tất cả các nút từ một nút nguồn duy nhất.
- **Tìm kiếm theo phương pháp tham lam:** Thuật toán này khám phá nút có vẻ gần mục tiêu nhất dựa trên hàm heuristic .

Nhờ vậy, A* vừa tối ưu vừa hiệu quả, và được đảm bảo sẽ tìm ra đường đi ngắn nhất nếu hàm heuristic thỏa mãn một số điều kiện nhất định.

5.3.2 Sử dụng A* Search cho TSP

Xét bài toán TSP với m thành phố, thuật toán A* sẽ không tìm kiếm trên đồ thị thành phố mà sẽ xây dựng các trạng thái và tìm kiếm trên đồ thị không gian trạng thái. Quy trình sử dụng A* để giải bài toán này như sau:

1. **Khởi tạo các tham số**: Nhận thông tin về vị trí của các thành phố $x_1, x_2, \ldots, x_m$ và ma trận khoảng cách $d(x_i, x_j) = \text{distance}(x_i, x_j)$, $\forall 1 \leq i, j \leq m$. Chọn một thành phố bắt đầu x_{root}.
2. **Biểu diễn trạng thái**: Mỗi trạng thái (node) trong không gian tìm kiếm của A* được biểu diễn bằng một cặp: $n = (x_i, P_{visited})$.
   - x_i: là thành phố hiện tại.
   - $P_{visited} = \{x_{p_1}, x_{p_2}, \ldots, x_{p_k}\}$: là một chuỗi (path) các thành phố đã đi qua theo đúng thứ tự.

   Số lượng trạng thái trong không gian này là $O(n * 2^n)$.
3. **Hàm đánh giá (Evaluation Function)**: A* sử dụng hàm $f(n) = g(n) + h(n)$ để đánh giá mỗi trạng thái.
   - **Hàm chi phí thực tế** $g(n)$: Là tổng chi phí thực tế của chu trình một phần từ thành phố bắt đầu đến thành phố hiện tại. Với trạng thái n có đường đi $P_{visited} = \{x_{p_1}, \ldots, x_{p_k}\}$, chi phí $g(n)$ được tính bằng:

$$g(n) = \sum_{i=1}^{k-1} d(x_{p_i}, x_{p_{i+1}}) \tag{31}$$

   - **Hàm heuristic** $h(n)$: Ước tính chi phí còn lại từ đỉnh hiện tại để hoàn thành chu trình. Để A* đảm bảo tìm ra giải pháp tối ưu toàn cục, heuristic $h(n)$ phải khả thi (admissible), nghĩa là nó không bao giờ ước tính quá cao chi phí thực tế còn lại ($h(n) \leq h^*(n)$). Một *Admissible Heuristic* hiệu quả phổ biến cho bài toán TSP là *Heuristic Cây Bao Trùm Nhỏ Nhất (MST)*[2], là tổng của:

[2]MST cung cấp một cận dưới (lower bound) cho độ dài của hành trình TSP tối ưu, do đó chi phí của MST được sử dụng làm Admissible Heuristic h(n) trong thuật toán A* cho TSP.

(a) Chi phí của Cây bao trùm tối thiểu (MST) trên tập các thành phố chưa được ghé thăm $V_{unvisited}$[3].

(b) Chi phí của cạnh ngắn nhất nối từ thành phố hiện tại (x_{p_k}) đến một thành phố trong $V_{unvisited}$.

(c) Chi phí của cạnh ngắn nhất nối từ thành phố bắt đầu (x_{p_1}) đến một thành phố trong $V_{unvisited}$.

Công thức có thể biểu diễn là:

$$h(n) = \text{MST_cost}(V_{unvisited}) + \min_{u \in V_{unvisited}} d(x_{p_k}, u) + \min_{v \in V_{unvisited}} d(x_{p_1}, v) \tag{32}$$

4. **Khởi tạo OPEN List**: Sử dụng một hàng đợi ưu tiên để lưu các trạng thái chưa được khám phá, sắp xếp theo giá trị $f(n)$. Thêm trạng thái khởi tạo $(x_{root}, \{x_{root}\})$ vào OPEN list.

5. **Vòng lặp tìm kiếm**:

   - Lấy trạng thái n có $f(n)$ nhỏ nhất từ OPEN list.
   - Nếu trạng thái n chứa tất cả m thành phố, tính tổng chi phí cuối cùng bằng cách cộng thêm khoảng cách quay về thành phố đầu tiên. Đây là giải pháp tối ưu.

$$\text{TotalCost} = g(n) + d(x_{current}, x_1) \tag{33}$$

   - Nếu không, tạo ra các trạng thái con bằng cách di chuyển từ thành phố hiện tại đến mỗi thành phố kề chưa được thăm.
   - Tính toán $f(n)$ cho các trạng thái con và thêm chúng vào OPEN list.

6. **Điều kiện dừng**: Khi tìm thấy giải pháp hoặc khi OPEN list rỗng.

5.4 Genetic Algorithm cho bài toán tối ưu hóa rời rạc

Giải thuật di truyền (*Genetic Algorithm* [14]) là một phương pháp tối ưu hóa sử dụng phương pháp ngẫu nhiên để tìm kiếm ngẫu nhiên các giải pháp tốt cho một bài toán cụ thể. *Genetic Algorithm* dựa trên học thuyết Darwin về "*sự sinh tồn của những cá thể khỏe mạnh nhất*", ngụ ý rằng những cá thể khỏe mạnh hơn có nhiều khả năng sống sót hơn và có nhiều cơ hội truyền lại những đặc điểm di truyền tốt của chúng cho thế hệ tiếp theo. Trong thuật toán di truyền, mỗi cá thể, tức là nhiễm sắc thể (*chromosome*) của một thành viên trong quần thể, đại diện cho một giải pháp tiềm năng cho bài toán.

5.4.1 Sử dụng Genetic Algorithm cho TSP

Xét bài toán *TSP* với m thành phố. Quy trình sử dụng *GA* [15] để giải bài toán này như sau:

1. Khởi tạo các siêu tham số: số thế hệ tối đa ($generation_{max}$), quy mô dân số (N), xác suất lai ghép ($p_{crossover}$), xác suất đột biến ($p_{mutation}$) và nhận thông tin về vị trí của các thành phố $x_1, x_2, \ldots, x_m$ và khoảng cách $d(x_i, x_j) = distance(x_i, x_j)$, $\forall 1 \leq i, j \leq m$.

[3]Thuật toán Prim là một thuật toán tham lam cổ điển tìm MST, hoạt động bằng cách xuất phát từ một đỉnh bất kì để xây dựng cây MST. Ở mỗi bước, Prim chọn cạnh có trọng số nhỏ nhất để nối một đỉnh có trong cây với một đỉnh ở ngoài cây.

2. Biểu diễn nhiễm sắc thể: $chromosome = X = \{x_1, x_2, \ldots, x_m\}$ là nhiễm sắc thể biểu diễn đường đi $x_1 \to x_2 \to \cdots \to x_m$.

3. Khởi tạo quần thể: Tạo quần thể gồm N cá thể (nhiễm sắc thể), mỗi nhiễm sắc thể đại diện cho một đường đi giữa các thành phố.

4. Hàm thích nghi (fitness function):

$$fitness(X) = \frac{1}{d(x_m, x_1) + \sum_{i=1}^{m-1} d(x_i, x_{i+1})} \tag{34}$$

5. Vòng lặp tiến hóa: Gồm lựa chọn, lai ghép và đột biến.

   - Lựa chọn (Tournament Selection): Chọn ngẫu nhiên k cá thể từ quần thể. Chọn ρ cá thể có fitness cao nhất "chiến thắng" và được chọn làm cha mẹ ($N > k > \rho$). Đặt $\rho = 2$ để đơn giản (ứng với 2 cha mẹ).
   - Lai ghép (Order Crossover): Chọn một đoạn con ngẫu nhiên từ X_{parent_1} sao chép vào X_{child} ở cùng vị trí. Điền các vị trí còn trống của X_{child} bằng các thành phố từ X_{parent_2}, theo thứ tự xuất hiện của chúng trong X_{parent_2}, nhưng bỏ qua những thành phố đã có trong đoạn con sao chép từ X_{parent_1}.
   - Đột biến: Hoán vị và đảo ngược nhiễm sắc thể trên các cá thể X_{child} với xác suất $p_{mutation}$.
     - Hoán vị: Chọn ngẫu nhiên hai vị trí (gen) trên nhiễm sắc thể và đổi chỗ.
     - Đảo ngược: Chọn một đoạn con ngẫu nhiên trên nhiễm sắc thể và đảo ngược thứ tự.

6. Vòng lặp tiến hóa sẽ dừng lại khi một trong các điều kiện sau được thỏa mãn:

   - $generation = generation_{max}$
   - $fitness(X)_{generation_{i+z}} - fitness(X)_{generation_i} \leq \epsilon$ với z là số thế hệ trải qua mà hàm $fitness(X)$ không cải thiện.
   - $fitness(X) \geq goal$

6 Thực nghiệm

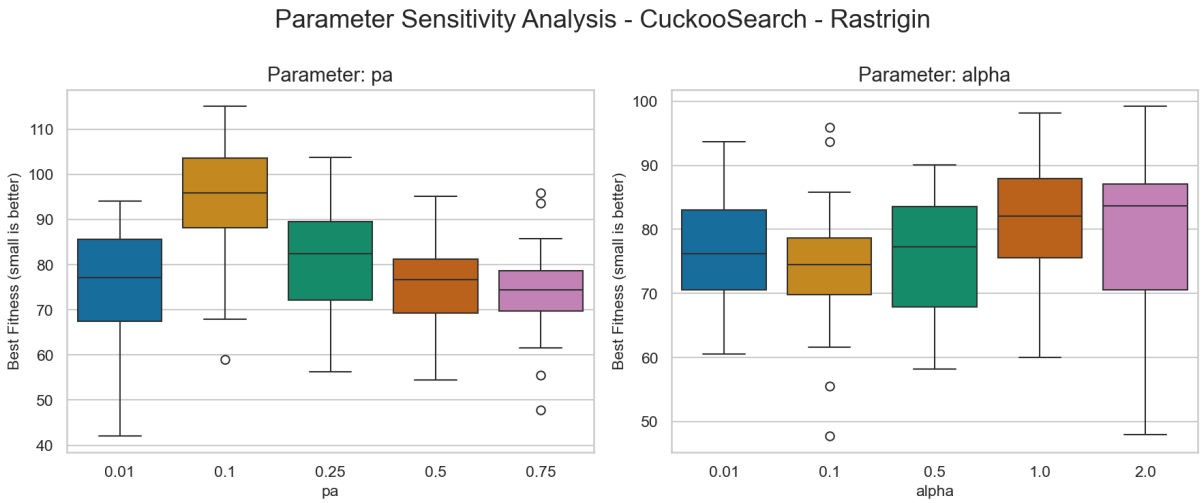
6.1 Tối ưu hóa liên tục

Phần này sẽ tiến hành phân tích độ nhạy của các tham số trong từng thuật toán. Tiếp theo, so sánh hiệu quả giữa các thuật toán trí tuệ bầy đàn dùng cho bài toán tối ưu hóa liên tục *(CS, ABC, FA)* và các thuật toán truyền thống *(SA, HC)*. Trong các thí nghiệm, số thế hệ (vòng lặp) được cố định là **100** và quy mô quần thể là **40**. Mỗi thử nghiệm sẽ được chạy lặp lại **30** lần để đảm bảo kết quả có ý nghĩa thống kê, do các thuật toán này đều mang tính ngẫu nhiên.

Trong quá trình thực nghiệm, nhóm đã xây dựng các hàm trực quan hóa trên không gian 2D (với nhiệt độ màu là các giá trị fitness) và trên không gian 3D để có thể quan sát, phân tích và đánh giá trạng thái các điểm dữ liệu của thuật toán tối ưu trên hàm benchmark Rastrigin và Sphere.

6.1.1 Cuckoo Search - CS

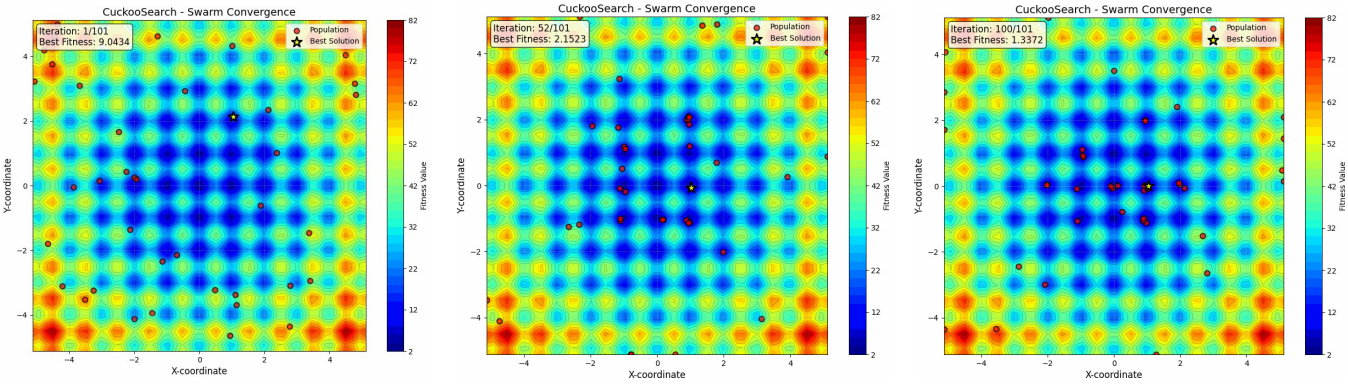
Sử dụng hàm benchmark Rastrigin, các tham số mặc định của thuật toán *Cuckoo Search* được thiết lập là $\alpha = 0.1$ và $p_a = 0.75$. Trong quá trình phân tích độ nhạy tham số, từng tham số sẽ được thay đổi riêng lẻ, trong khi tham số còn lại được giữ cố định nhằm đánh giá độ nhạy của từng tham số đến hiệu quả của thuật toán.



Hình 4: Độ nhạy tham số với thuật toán CS

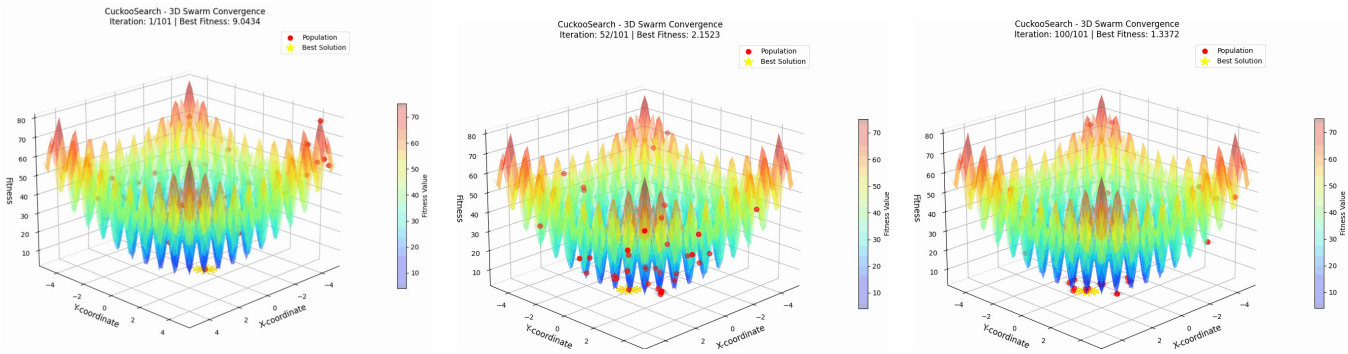
Đối với tham số p_a, giá trị $p_a = 0.75$ cho thấy hiệu suất tối ưu nhất, thể hiện qua việc hộp (IQR) nằm ở vị trí thấp và có biên độ hẹp, cho thấy kết quả vừa tốt vừa ổn định. Đối với tham số alpha, hiệu suất thuật toán có xu hướng giảm khi alpha tăng (các giá trị 1.0 và 2.0 có fitness cao hơn). Các giá trị thấp như $\alpha = 0.1$ và $\alpha = 0.5$ cho kết quả tốt hơn; trong đó $\alpha = 0.1$ đạt được mức trung vị thấp nhất nhưng cũng xuất hiện nhiều điểm ngoại lệ, cho thấy khả năng thiếu ổn định, trong khi $\alpha = 0.5$ cung cấp một sự cân bằng tốt.

Với hàm benchmark Rastrigin 2 chiều, cùng với các thông số mặc định $\alpha = 0.1$ và $p_a = 0.75$ của thuật toán Cuckoo Search, quá trình hội tụ diễn ra như sau:



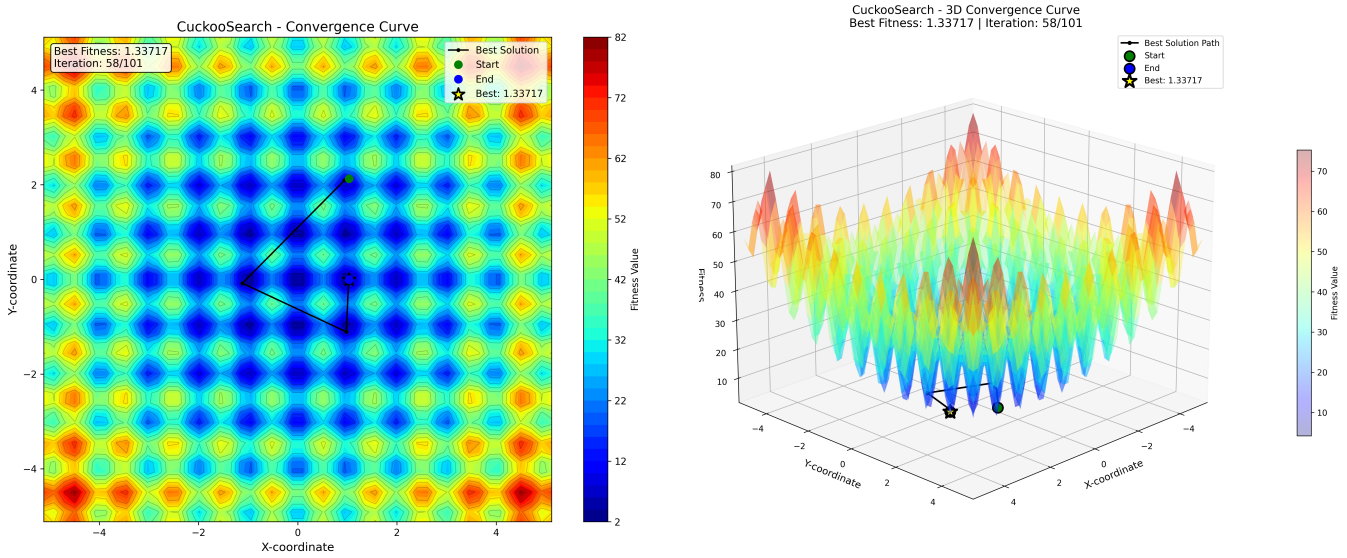
(a) Giai đoạn đầu (b) Giai đoạn giữa (c) Quá trình cuối

Hình 5: Quá trình hội tụ của Cuckoo Search trên hàm Rastrigin (Không gian 2D)



(a) Giai đoạn đầu (b) Giai đoạn giữa (c) Giai đoạn cuối

Hình 6: Quá trình hội tụ của Cuckoo Search trên hàm Rastrigin (Không gian 3D)



(a) Biểu đồ hội tụ (2D) (b) Biểu đồ hội tụ (3D)

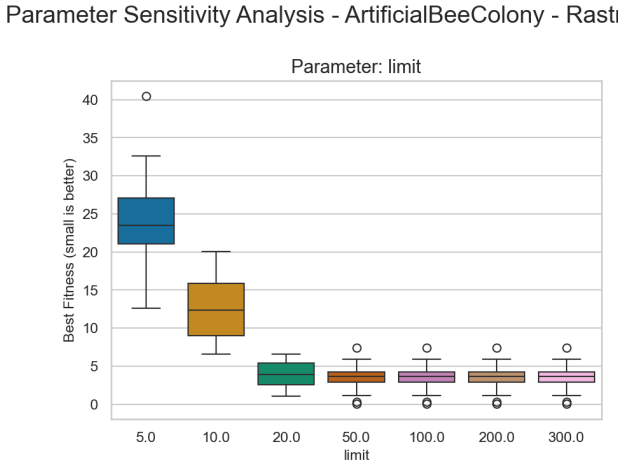
Hình 7: Quá trình tìm kiếm nghiệm tốt nhất của Cuckoo Search trên hàm Rastrigin

Dựa vào trực quan hóa, quá trình hội tụ của thuật toán Cuckoo Search đạt hiệu quả cao trong việc khám phá không gian tìm kiếm.

- Giai đoạn đầu các quần thể được phân tán ngẫu nhiên, khám phá trên toàn bộ không gian tìm kiếm.
- Giai đoạn giữa, thuật toán bắt đầu hội tụ, các quần thể tập trung ở nhiều thung lũng (điểm cực tiểu cục bộ) khác nhau. Điều này trực quan hóa đặc điểm cốt lõi của Cuckoo Search là việc sử dụng cơ chế Lévy Flights, giúp thuật toán thoát khỏi các điểm tối ưu cục bộ và khám phá không gian tìm kiếm một cách hiệu quả, thay vì chỉ tập trung vào một điểm.
- Giai đoạn cuối, các quần thể hội tụ mạnh mẽ về điểm trung tâm (cực tiểu toàn cục), cho thấy Cuckoo Search tính ổn định cao qua việc thuật toán nhất quán tìm được các giá trị fitness cuối cùng rất thấp và có độ phân tán rất nhỏ.

6.1.2 Artificial Bee Colony - ABC

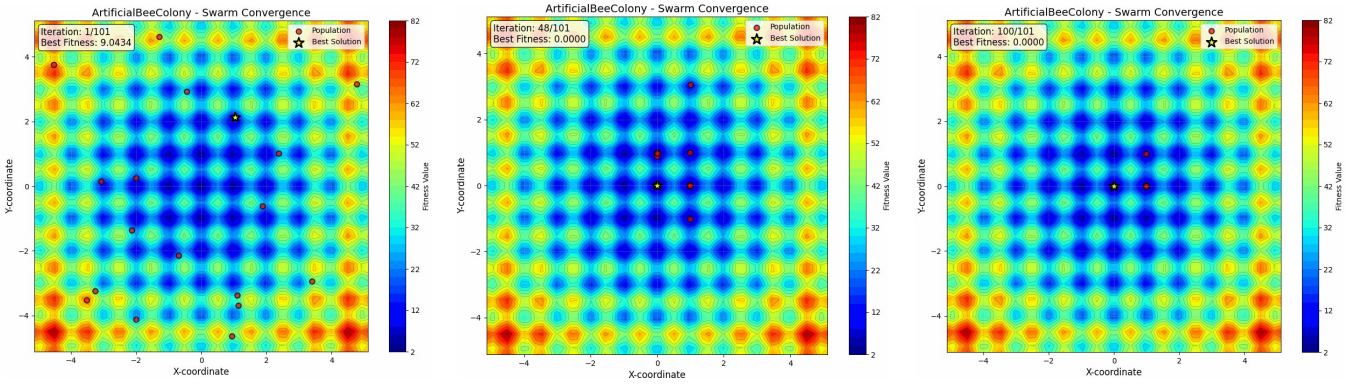
Sử dụng hàm benchmark Rastrigin, các tham số mặc định của thuật toán ABC được thiết lập là $limit = \lfloor \frac{N}{2} \rfloor * D = 200$ với $N = 40$ là quy mô quần thể và $D = 10$ là số chiều của benchmark. Tham số $limit$ sẽ được đặt lần lượt bằng các kích thước cố định để đo độ nhạy của tham số.



Hình 8: Độ nhạy tham số với thuật toán ABC

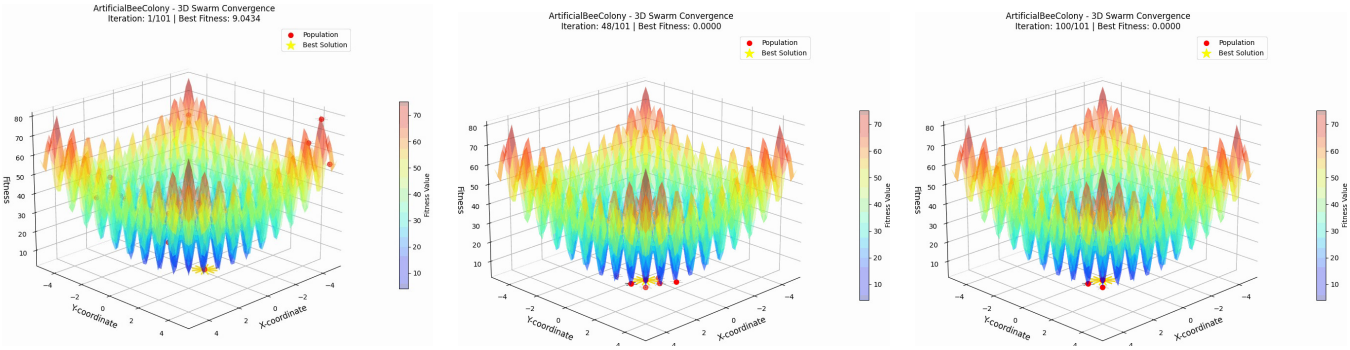
Biểu đồ cho thấy thuật toán Artificial Bee Colony (ABC) thể hiện độ nhạy cao với tham số *limit* ở các giá trị thấp. Khi *limit* tăng từ 5 lên 20, hiệu suất được cải thiện đáng kể, với giá trị trung vị giảm mạnh từ trên 20 xuống dưới 5. Tuy nhiên, khi *limit* tiếp tục tăng từ 20 đến 300, thuật toán trở nên ít nhạy cảm hơn. Trong khoảng giá trị này, hiệu suất duy trì ở mức rất tốt và ổn định, thể hiện qua các hộp (IQR) hẹp và nằm sát mức fitness thấp, cho thấy chỉ cần một giá trị limit đủ lớn là thuật toán đã đạt được hiệu suất tối ưu và ổn định.

Với hàm benchmark Rastrigin 2 chiều, cùng với các thông số mặc định $limit = 40$ của thuật toán Artificial Bee Colony, quá trình hội tụ diễn ra như sau:



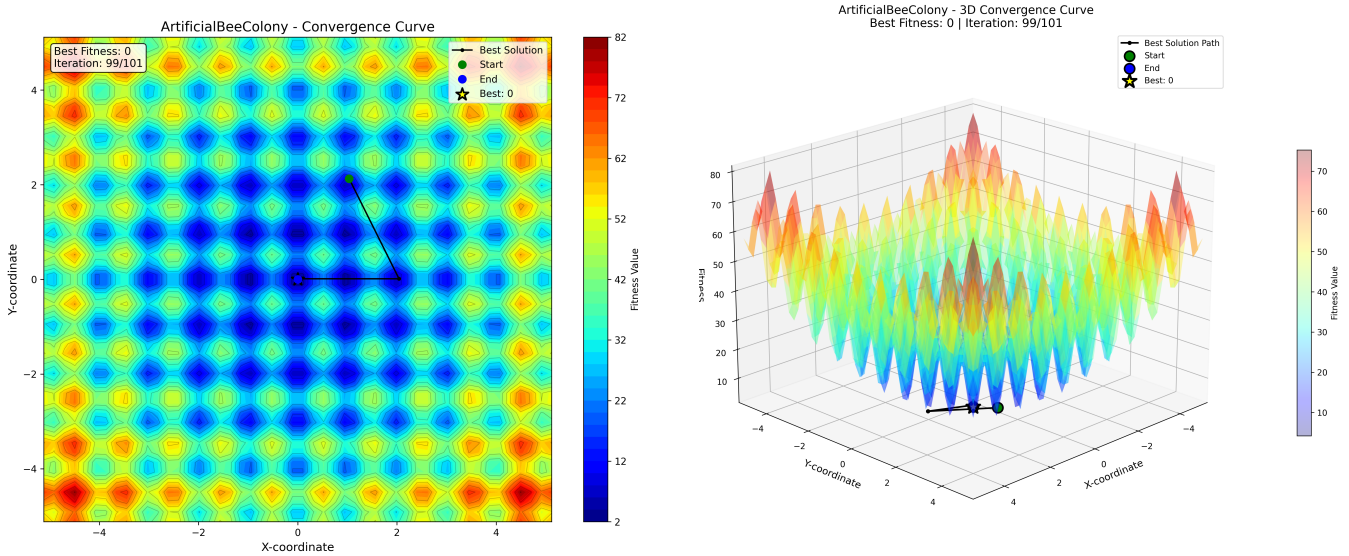
(a) Giai đoạn đầu (b) Giai đoạn giữa (c) Quá trình cuối

Hình 9: Quá trình hội tụ của Artificial Bee Colony trên hàm Rastrigin (Không gian 2D)



(a) Giai đoạn đầu (b) Giai đoạn giữa (c) Giai đoạn cuối

Hình 10: Quá trình hội tụ của Artificial Bee Colony trên hàm Rastrigin (Không gian 3D)



(a) Biểu đồ hội tụ (2D) (b) Biểu đồ hội tụ (3D)

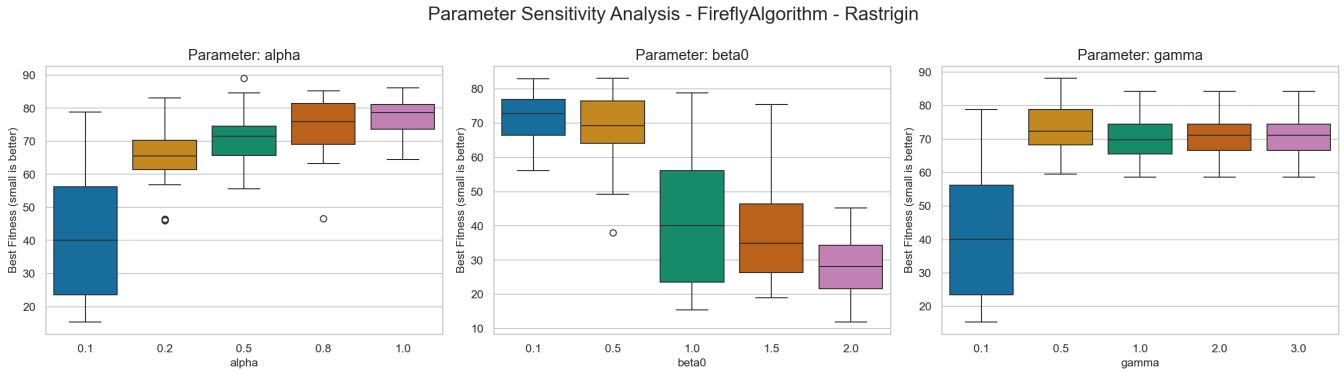
Hình 11: Quá trình tìm kiếm nghiệm tốt nhất của Artificial Bee Colony trên hàm Rastrigin

Dựa vào trực quan hóa, quá trình hội tụ của thuật toán Artificial Bee Colony được kiểm soát bởi sự cân bằng giữa khai thác và khám phá thông qua ba nhóm ong.

- Giai đoạn đầu các nguồn thức ăn được phân tán ngẫu nhiên.
- Giai đoạn giữa, thuật toán hội tụ rất nhanh, bầy ong đã xác định được khu vực trung tâm (cực tiểu toàn cục) có nguồn thức ăn "chất lượng" nhất và tập trung khai thác mạnh mẽ tại đó.
- Giai đoạn cuối, thuật toán thể hiện khả năng hội tụ vượt trội cho thấy cho thấy Artificial Bee Colony cân bằng hiệu quả giữa các pha (ong thợ, ong quan sát, và ong trinh sát) giúp nhanh chóng loại bỏ các giải pháp kém và tập trung vào giải pháp tốt nhất.

6.1.3 Firefly Algorithm - FA

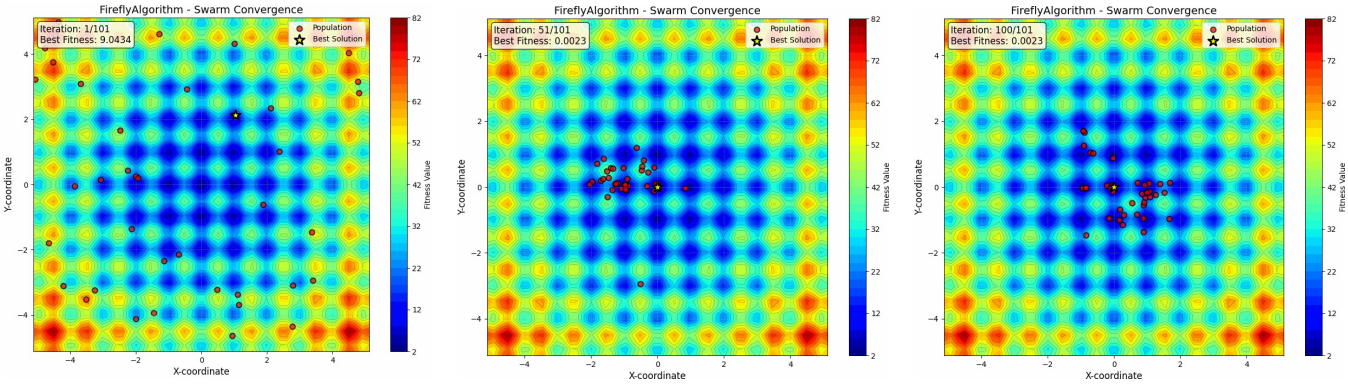
Sử dụng hàm benchmark Rastrigin, các tham số mặc định của thuật toán *FA* được thiết lập là $\alpha = 0.1$, $\beta_0 = 2$ và $\gamma = 0.1$. Trong quá trình phân tích độ nhạy tham số, từng tham số sẽ được thay đổi riêng lẻ, trong khi tham số còn lại được giữ cố định nhằm đánh giá độ nhạy của từng tham số đến hiệu quả của thuật toán.



Hình 12: Độ nhạy tham số với thuật toán FA

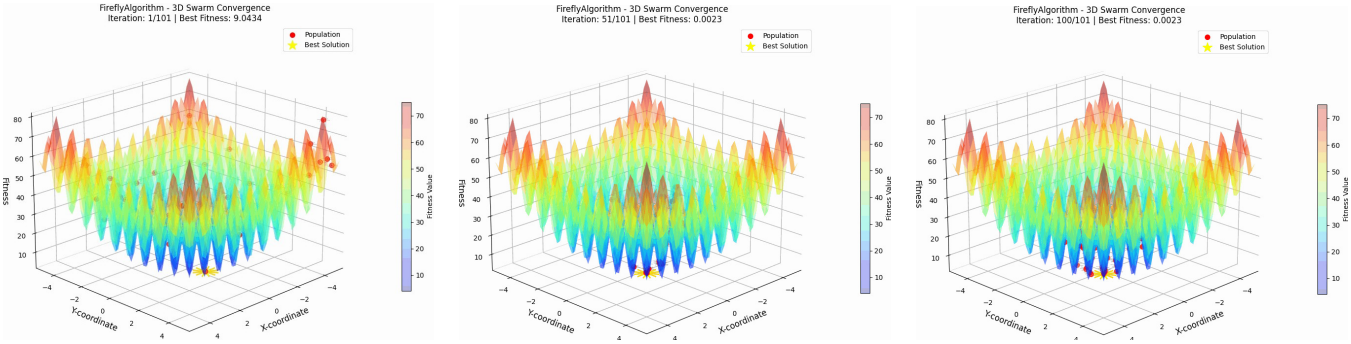
Biểu đồ cho thấy Thuật toán *Firefly* cực kỳ nhạy cảm với cả ba tham số α, β_0 và γ trên hàm Rastrigin. Đối với alpha và gamma, hiệu suất tối ưu đạt được rất rõ rệt ở giá trị thấp nhất được thử nghiệm, là $\alpha = 0.1$ và $\gamma = 0.1$; hiệu suất suy giảm nghiêm trọng ngay khi các giá trị này tăng lên. Ngược lại, thuật toán thể hiện xu hướng hoàn toàn đối lập với $beta_0$: hiệu suất rất kém ở các giá trị $beta_0$ thấp và cải thiện một cách tuyến tính khi $beta_0$ tăng, trong đó giá trị $beta_0 = 2.0$ mang lại kết quả vừa tốt nhất vừa ổn định nhất.

Với hàm benchmark Rastrigin 2 chiều, cùng với các thông số mặc định $\alpha = 0.1$, $\beta_0 = 2$ và $\gamma = 0.1$ của thuật toán Firefly Algorithm, quá trình hội tụ diễn ra như sau:



(a) Giai đoạn đầu (b) Giai đoạn giữa (c) Quá trình cuối

Hình 13: Quá trình hội tụ của Firefly Algorithm trên hàm Rastrigin (Không gian 2D)

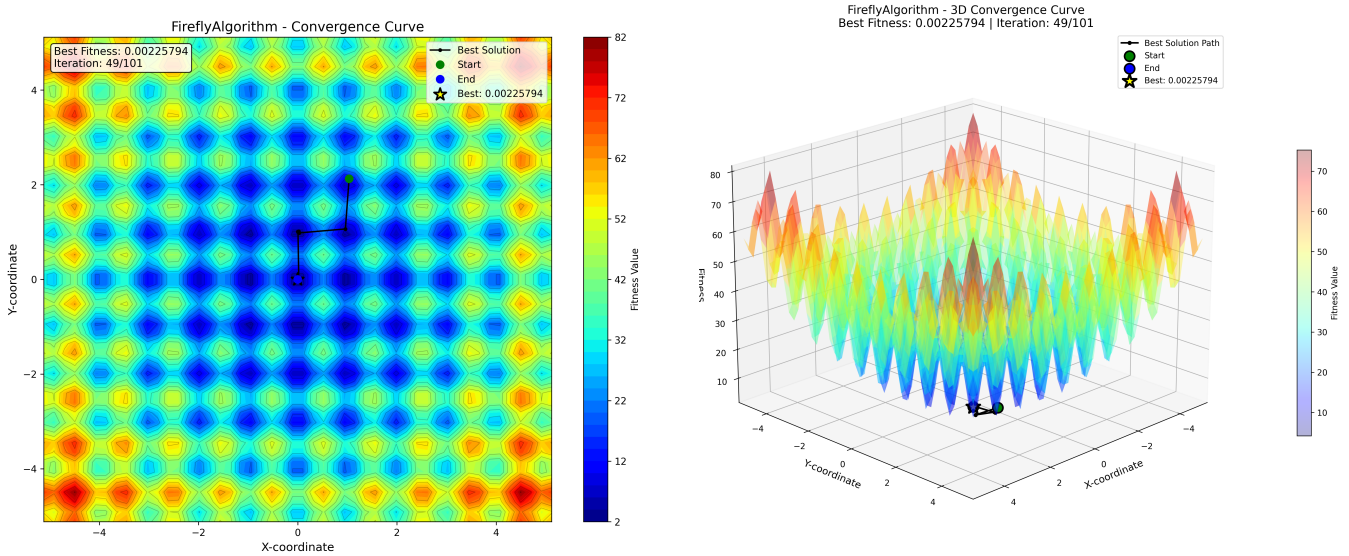


(a) Giai đoạn đầu (b) Giai đoạn giữa (c) Giai đoạn cuối

Hình 14: Quá trình hội tụ của Firefly Algorithm trên hàm Rastrigin (Không gian 3D)

Dựa vào trực quan hóa, quá trình hội tụ của thuật toán Firefly Algorithm diễn ra rất nhanh chóng xung quanh điểm tối ưu toàn cục.

- Giai đoạn đầu, các quần thể (đom đóm) được phân tán ngẫu nhiên.
- Giai đoạn giữa, cơ chế hấp dẫn của thuật toán được thể hiện rất rõ. Các đom đóm ở khu vực "ít sáng" (fitness cao) bị thu hút và di chuyển rõ rệt về phía các đom đóm "sáng hơn" (fitness thấp) đang tập trung ở trung tâm.
- Giai đoạn cuối, thuật toán thể hiện khả năng khai thác và hội tụ xuất sắc, cho thấy tính ổn định cao thể hiện qua sự nhất quán trong việc tìm kiếm các giá trị fitness cuối cùng rất thấp và độ phân tán nhỏ
- Đường đi của giải pháp tốt nhất cho thấy nó di chuyển thẳng về khu vực trung tâm và hội tụ tại đó, phản ánh cơ chế bị thu hút bởi giải pháp sáng (tốt) hơn.



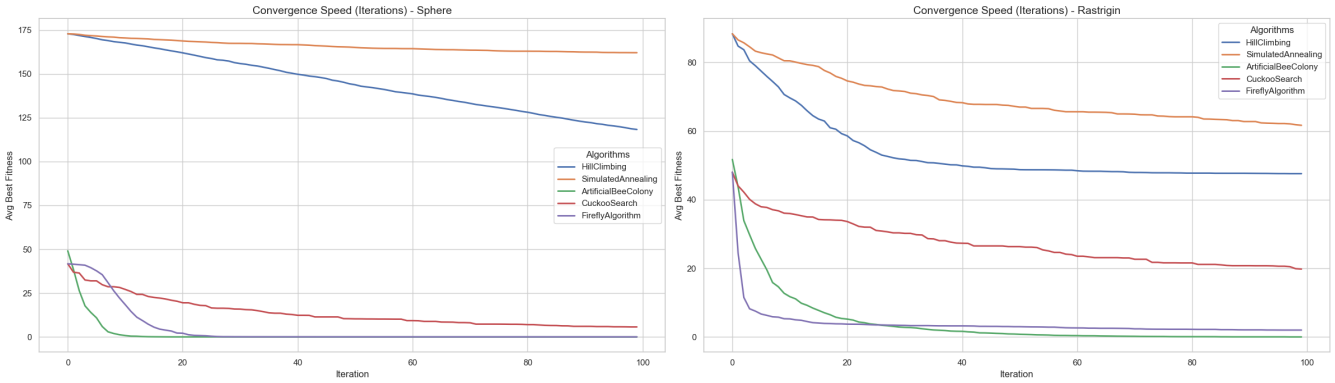
(a) Biểu đồ hội tụ (2D) (b) Biểu đồ hội tụ (3D)

Hình 15: Quá trình tìm kiếm nghiệm tốt nhất của Firefly Algorithm trên hàm Rastrigin

6.1.4 So sánh giải thuật bầy đàn và giải thuật truyền thống

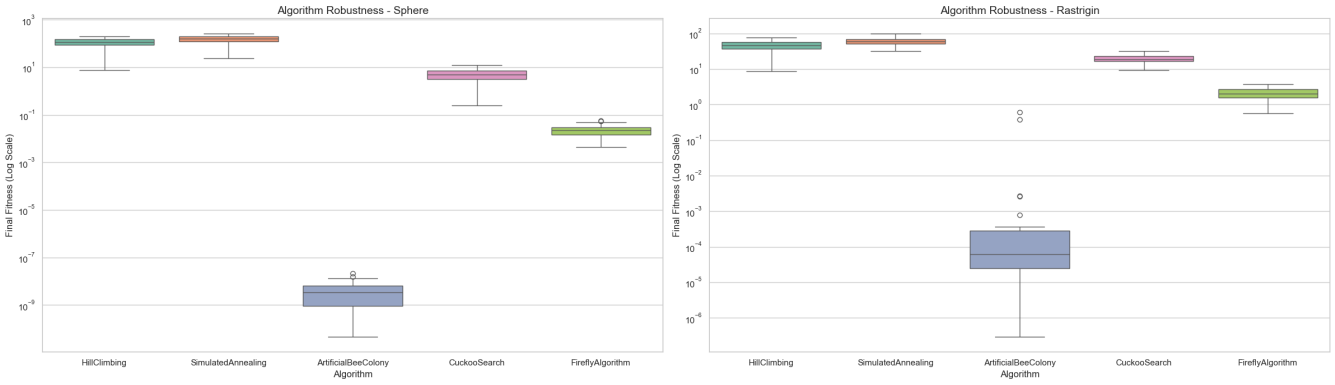
Phần này sẽ tiến hành so sánh hiệu quả giữa các thuật toán trí tuệ bầy đàn *(CS, ABC, FA)* và các thuật toán truyền thống *(SA, HC)* trên bài toán tối ưu hóa liên tục với cả hai hàm benchmark. Các thuật toán bầy đàn sẽ sử dụng tham số mặc định. Tham số của thuật toán *HC* sẽ được thiết lập là $stepsize = 0.1$ và tham số của thuật toán *SA* sẽ được thiết lập là $stepsize = 0.1, T_{max} = 1000, \alpha = 0.95$.

Tốc độ hội tụ: Các thuật toán bầy đàn *(CS, FA, ABC)* cho thấy tốc độ hội tụ vượt trội so với các thuật toán truyền thống *(SA, HC)*. Trên cả hai hàm benchmark, CS và FA hội tụ nhanh nhất, đạt được giá trị fitness tối ưu (gần 0) chỉ trong số ít lần lặp. Ngược lại, SA và HC hội tụ rất chậm và bị kẹt ở các giá trị fitness cao, không tìm được lời giải tốt ngay cả sau 100 lần lặp.



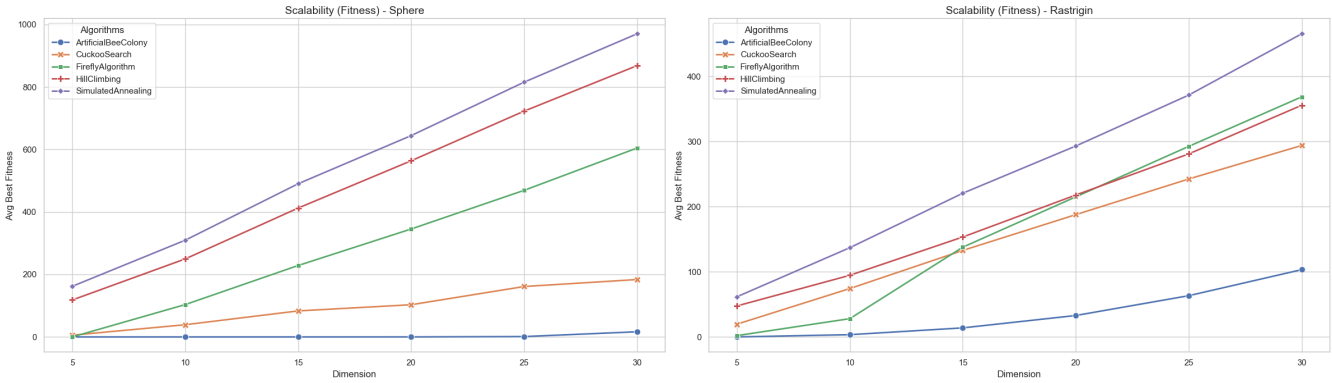
Hình 16: So sánh tốc độ hội tụ (theo lần lặp).

Tính ổn định: Biểu đồ hộp cho thấy *FA* và *CS* có tính ổn định cao nhất, thể hiện qua việc chúng nhất quán tìm được các giá trị fitness cuối cùng rất thấp và có độ phân tán rất nhỏ. *ABC* kém ổn định hơn, đặc biệt trên hàm Rastrigin khi có nhiều giá trị ngoại lai nhưng lại có kết quả hội tụ tốt nhất. *HC* và *SA* cũng ổn định, nhưng ở mức fitness rất cao, cho thấy chúng liên tục hội tụ về các lời giải không tốt.



Hình 17: So sánh tính ổn định của các thuật toán.

Khả năng mở rộng: *ABC* thể hiện khả năng mở rộng tốt nhất; chất lượng lời giải của nó ít bị suy giảm nhất khi số chiều (độ phức tạp) của bài toán tăng lên, đặc biệt là trên hàm Sphere. *FA* và *CS* cũng có khả năng mở rộng tốt hơn đáng kể so với các thuật toán truyền thống. Tuy nhiên, thuật toán *FA* lại cho thấy sự thua kém so với *HC* ở hàm phức tạp như astrigin khi độ lớn bài toán tăng cao. *HC* và *SA* cho thấy khả năng mở rộng kém nhất, khi chất lượng lời giải suy giảm rất nhanh khi số chiều tăng.

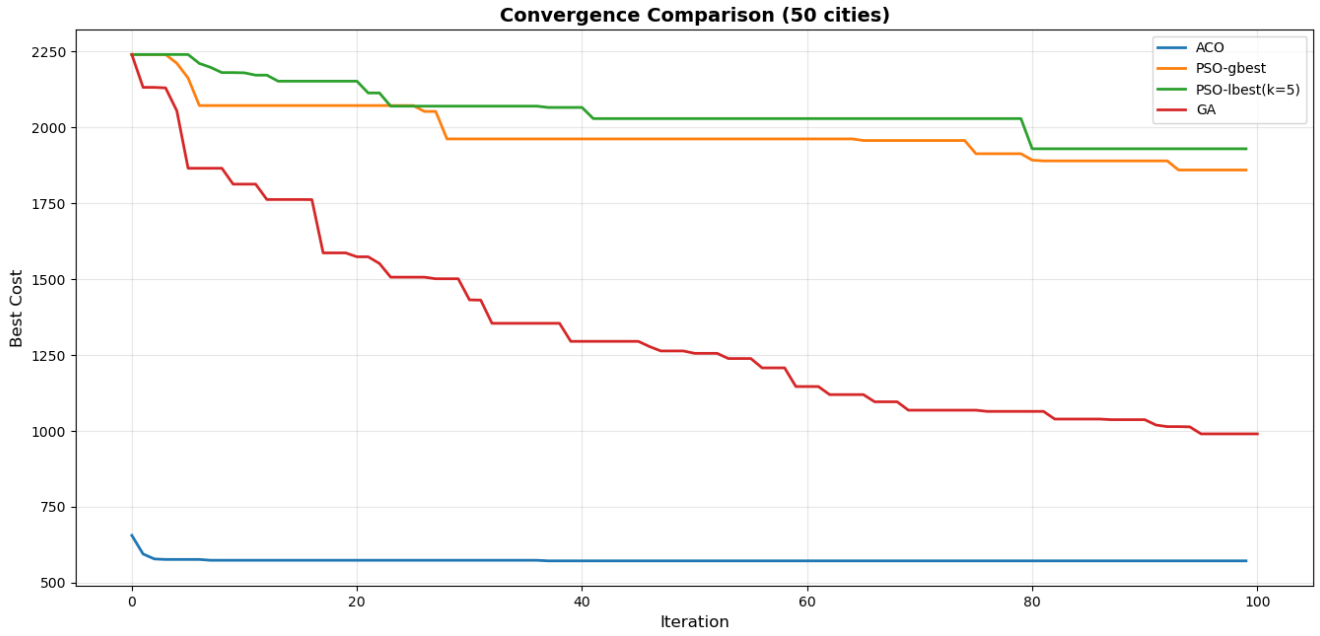


Hình 18: So sánh khả năng mở rộng của các thuật toán.

6.2 Tối ưu hóa rời rạc

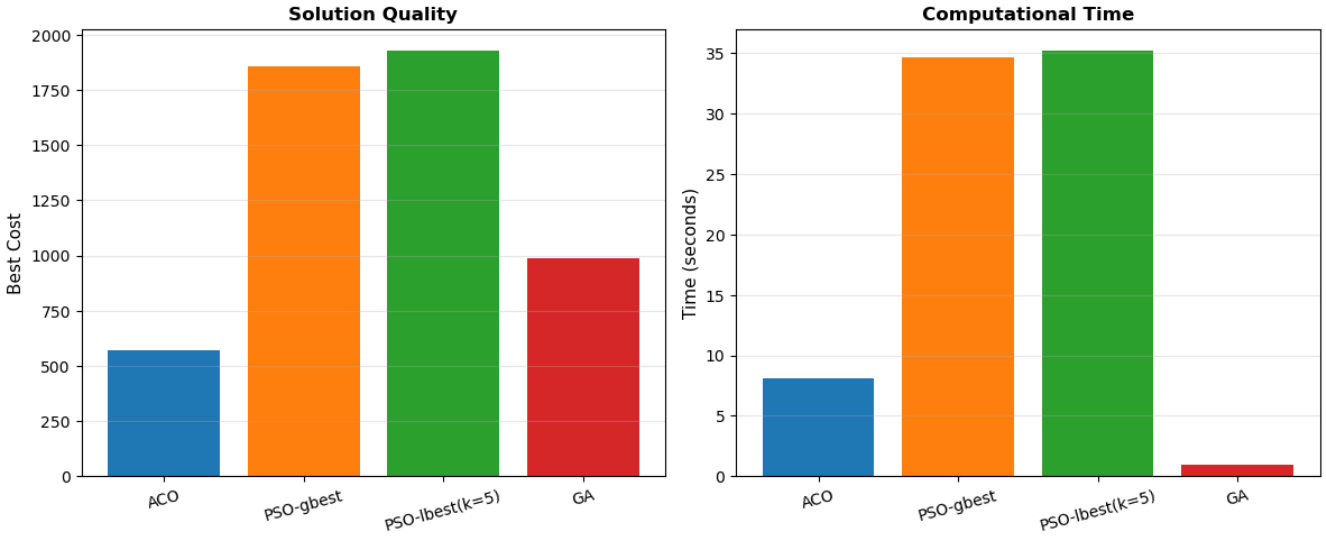
Chúng ta sẽ so sánh các giải thuật trí tuệ bầy đàn như *ACO*, *PSO* với các thuật toán truyền thống như *GA* và *A** trên bài toán tối ưu hóa rời rạc *TSP*. Vì *A** không chạy dựa trên vòng lặp tối ưu hóa và tạo các cá thể, chúng ta sẽ xét riêng thuật toán này sau. Xét bài toán *TSP* với 50 thành phố, các giải thuật *ACO*, *PSO* và *GA* chạy 100 vòng lặp, khởi tạo 50 cá thể:

Tốc độ hội tụ: Tốc độ hội tụ *ACO* là tốt nhất, sau đó đến *GA*. *gbest-PSO* và *lbest-PSO* có tốc độ hội tụ gần như tương đương nhau.



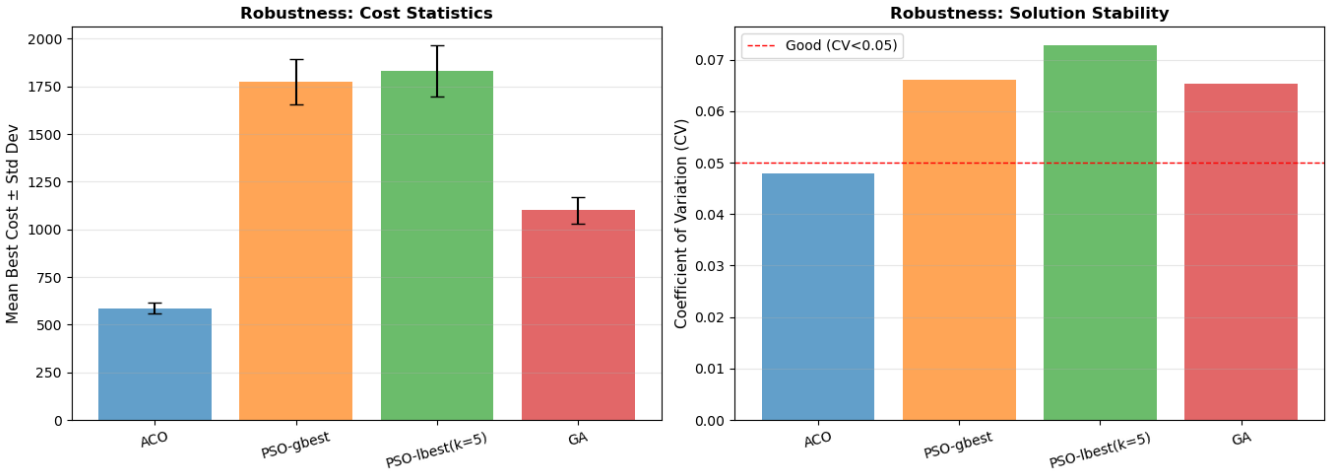
Hình 19: So sánh tốc độ hội tụ trên TSP

Chất lượng giải thuật và độ phức tạp thời gian: Chất lượng lời giải của *ACO* là tốt nhất, *GA* có tốc độ nhanh nhất và lời giải ở mức chấp nhận được. Còn với *gbest-PSO* và *lbest-PSO*, tốc độ chậm và lời giải cũng không tối ưu.



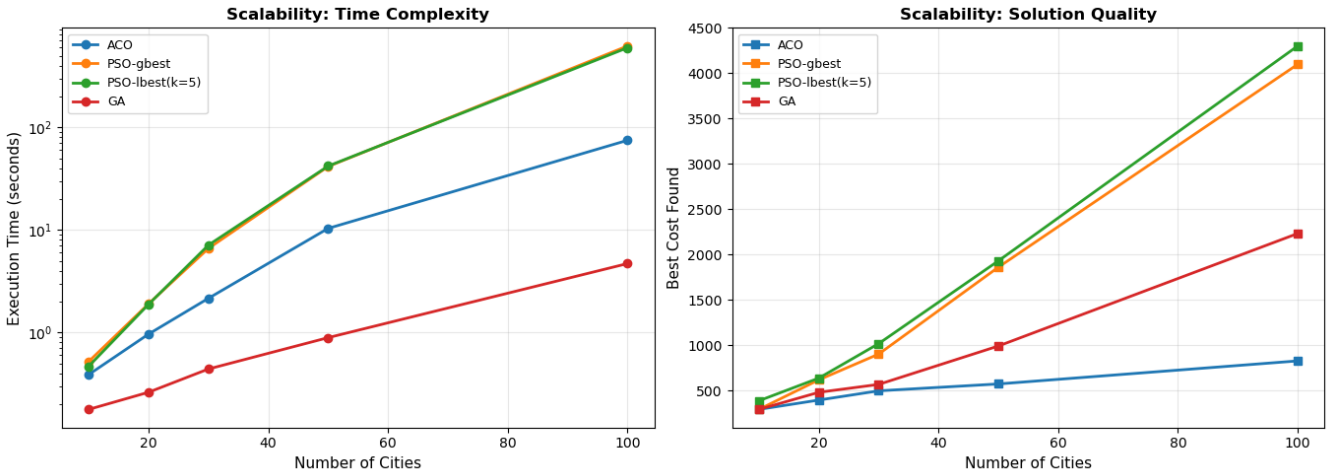
Hình 20: So sánh chất lượng giải thuật và độ phức tạp thời gian trên TSP

Tính ổn định của giải thuật: Với 10 lần chạy giải thuật, khoảng tứ phân vị (IQR) của *ACO* là nhỏ nhất. Hệ số biến thiên của các giải thuật theo thứ tự nhỏ đến lớn là *ACO* ($CV_{ACO} < 0.05$), *GA*, *lbest-PSO*, *gbest-PSO*.



Hình 21: So sánh tính ổn định của các giải thuật trên TSP

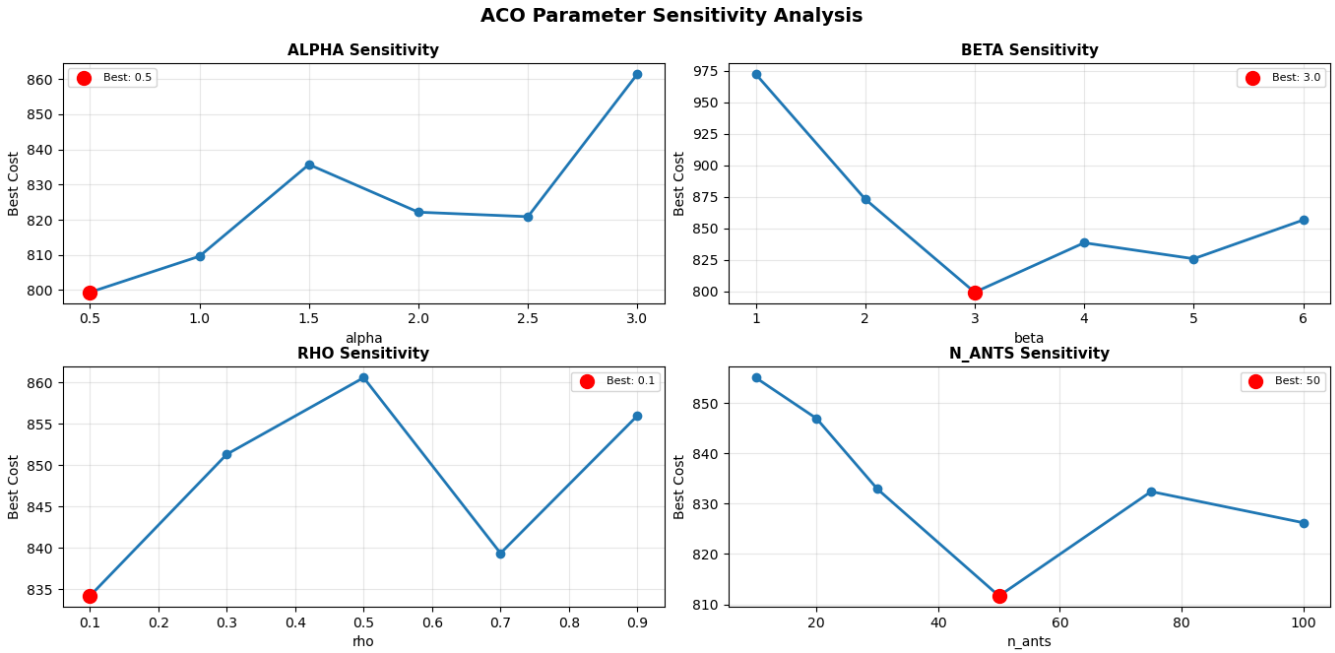
Khả năng mở rộng của giải thuật: Với *TSP* số lượng thành phố $m \in \{10, 20, 30, 50, 100\}$, số lượng vòng lặp được điều chỉnh phù hợp: $n_{iteration} = min(200, max(50, 2 \cdot m)$. Với m tăng, *ACO* vẫn giữ chất lượng lời giải tốt. *lbest-PSO* và *gbest-PSO* đều giảm chất lượng lời giải và có thời gian tính toán chậm hơn. *GA* vẫn có thời gian tính toán nhanh, nhưng lời giải không tốt bằng *ACO* khi m lớn.



Hình 22: So sánh khả năng mở rộng của các giải thuật trên TSP

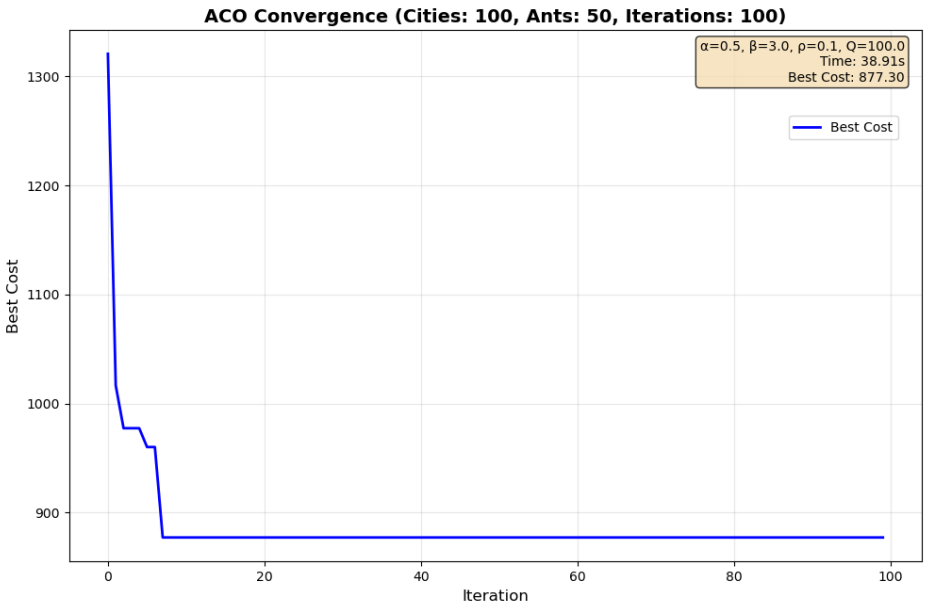
6.2.1 Ant Colony Optimization - ACO

Xét *TSP* $m = 100$ thành phố, các tham số mặc định cho *ACO* là $\alpha = 2.0$, $\beta = 5.0$, $\rho = 0.5$, $Q = 100$, $n_{ants} = 30$, $n_{iteration} = 100$, lần lượt thay đổi một tham số và cố định các tham số còn lại để đo độ nhạy (sự ảnh hưởng) của tham số đối với lời giải:

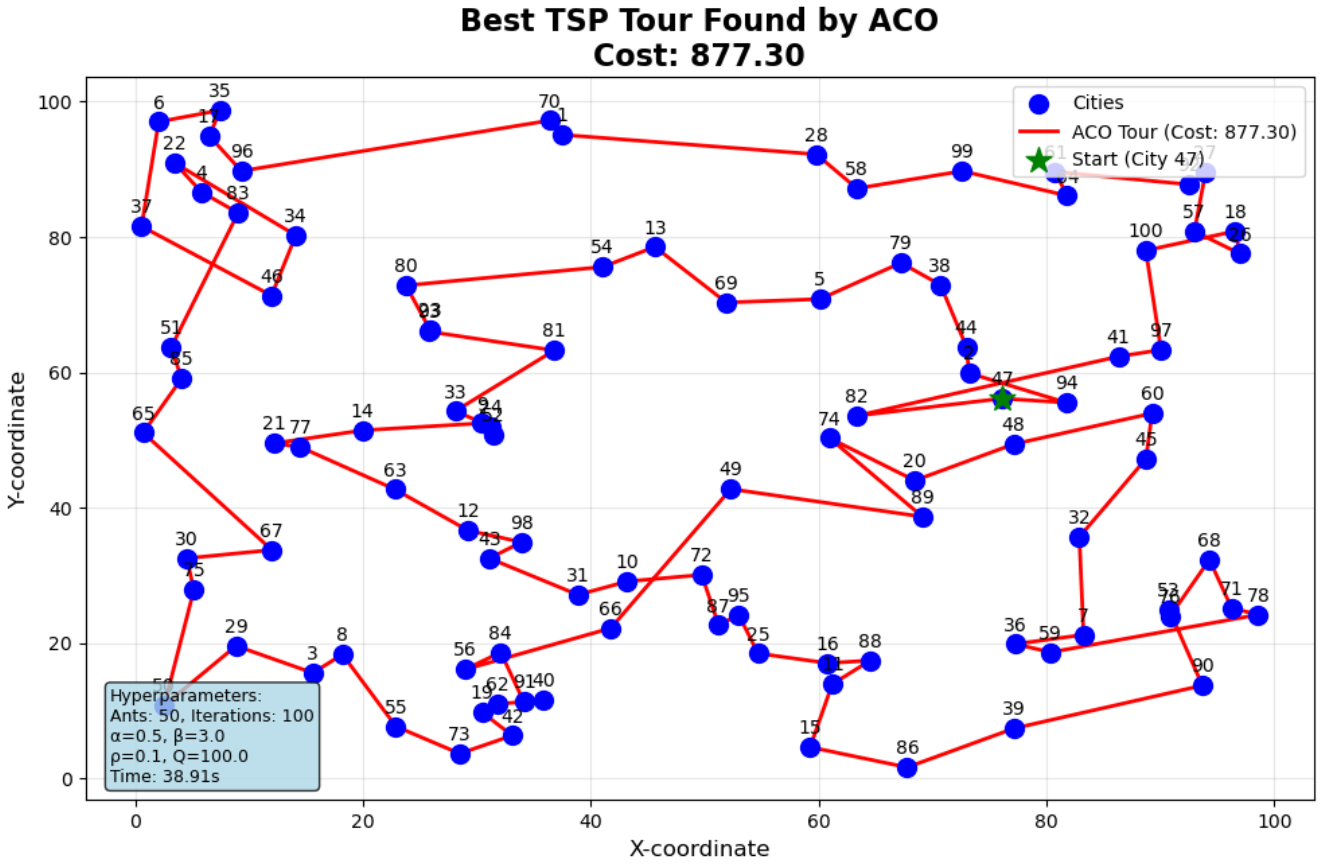


Hình 23: Độ nhạy tham số với thuật toán ACO

Biểu đồ cho thấy các tham số $\alpha = 0.5$, $\beta = 3.0$, $\rho = 0.1$, $n_{ants} = 50$ là tối ưu nhất (trường hợp chỉ thay một tham số so với tham số mặc định). Tuy nhiên, các tham số này không ổn định nên không có quy luật cụ thể cho việc chọn tham số tối ưu cho mọi trường hợp. Điều này chứng minh bằng việc kết hợp các tham số trên, với $m = 100$, $n_{iteration} = 100$ thì *best_cost* vẫn không là tối ưu nhất.



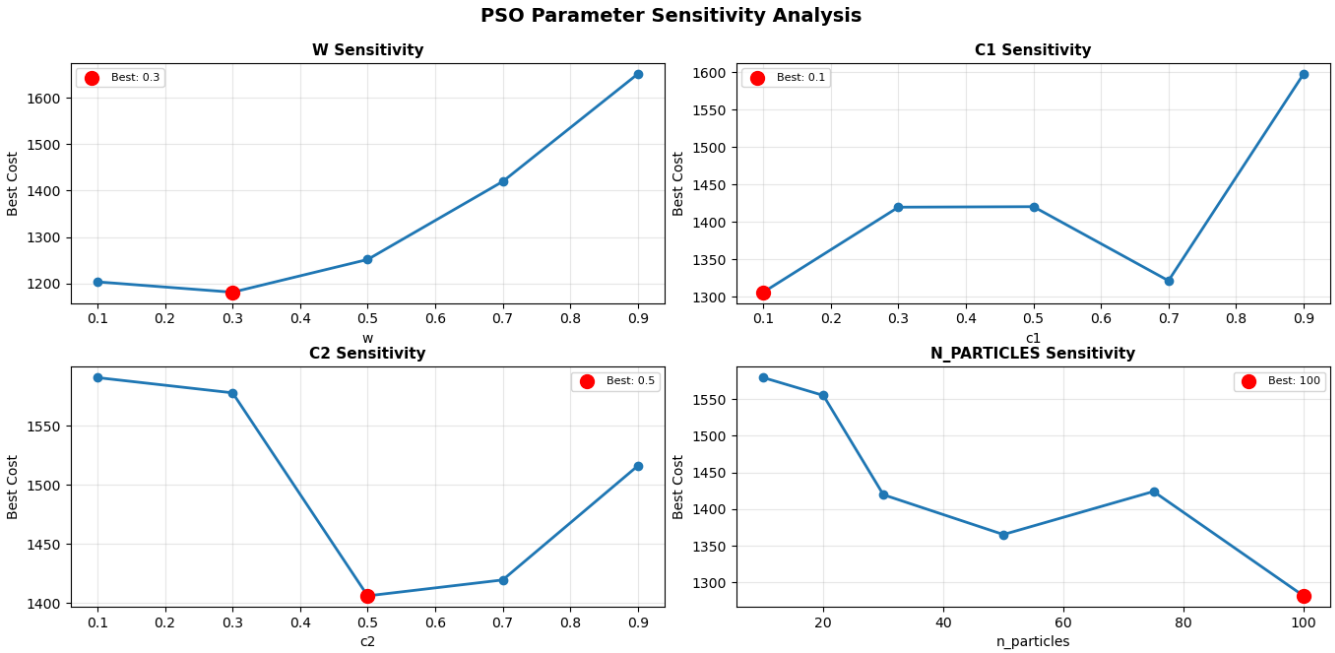
Hình 24: Tốc độ hội tụ của ACO



Hình 25: Đường đi Hamilton tìm bởi ACO cho TSP

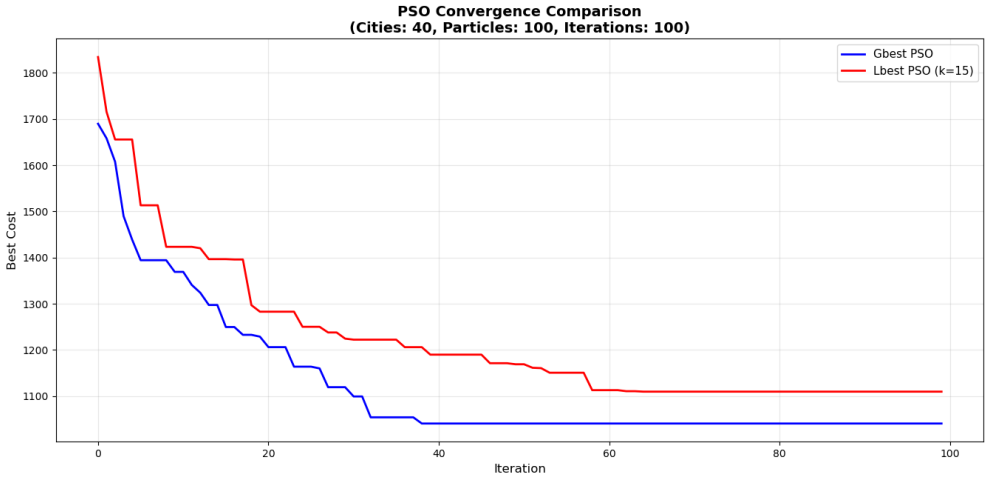
6.2.2 Particle Swarm Optimization - PSO

Xét *TSP* $m = 40$ thành phố, các tham số mặc định cho *PSO* (topology gbest) là $w = 0.3$, $c_1 = 0.3$, $c_2 = 0.7$, $n_{particles} = 30$, $n_{iteration} = 100$, lần lượt thay đổi một tham số và cố định các tham số còn lại để đo độ nhạy (sự ảnh hưởng) của tham số đối với lời giải:

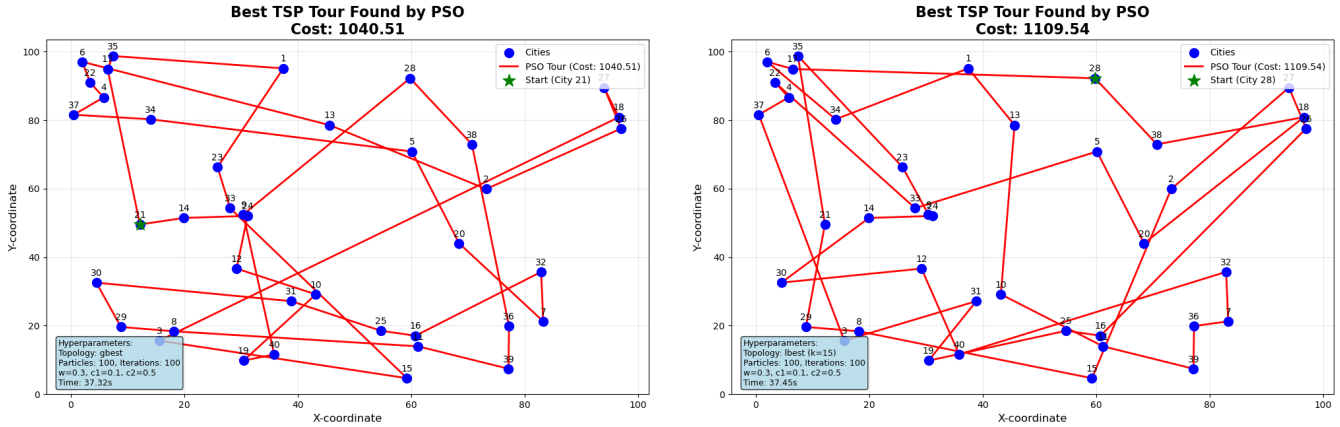


Hình 26: Độ nhạy tham số với thuật toán PSO

Biểu đồ cho thấy các tham số $w = 0.3$, $c_1 = 0.1$, $c_2 = 0.5$, $n_{particles} = 100$ là tối ưu nhất (trường hợp chỉ thay một tham số so với tham số mặc định). Có thể thấy rằng w và c_1 ở ngưỡng thấp, c_2 ở mức trung bình và $n_{particles}$ cao thì thuật toán chạy ra kết quả tối ưu. Ta sẽ sử dụng $w = 0.3$, $c_1 = 0.1$, $c_2 = 0.5$, $n_{particles} = 100$, $n_{interation} = 100$ để chạy *gbest PSO* và *lbest PSO*.



Hình 27: Tốc độ hội tụ của gbest-PSO và lbest-PSO

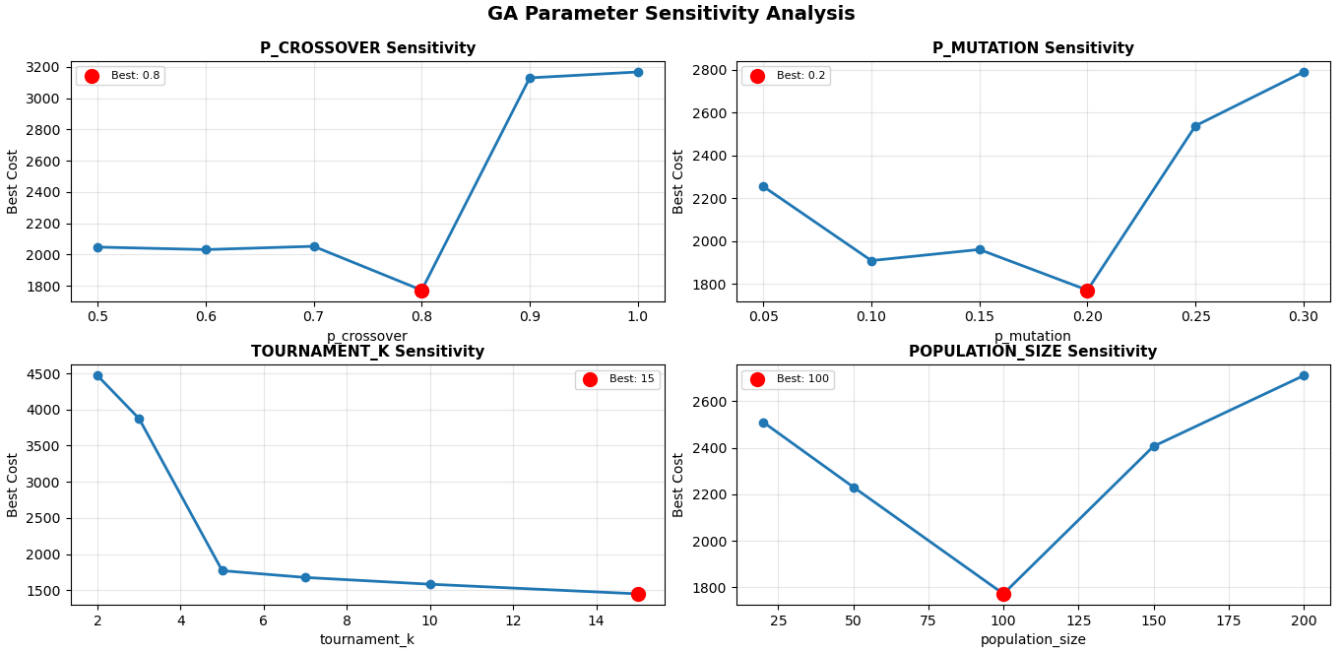


(a) gbest-PSO (b) lbest-PSO

Hình 28: Đường đi Hamilton tìm bởi PSO cho TSP

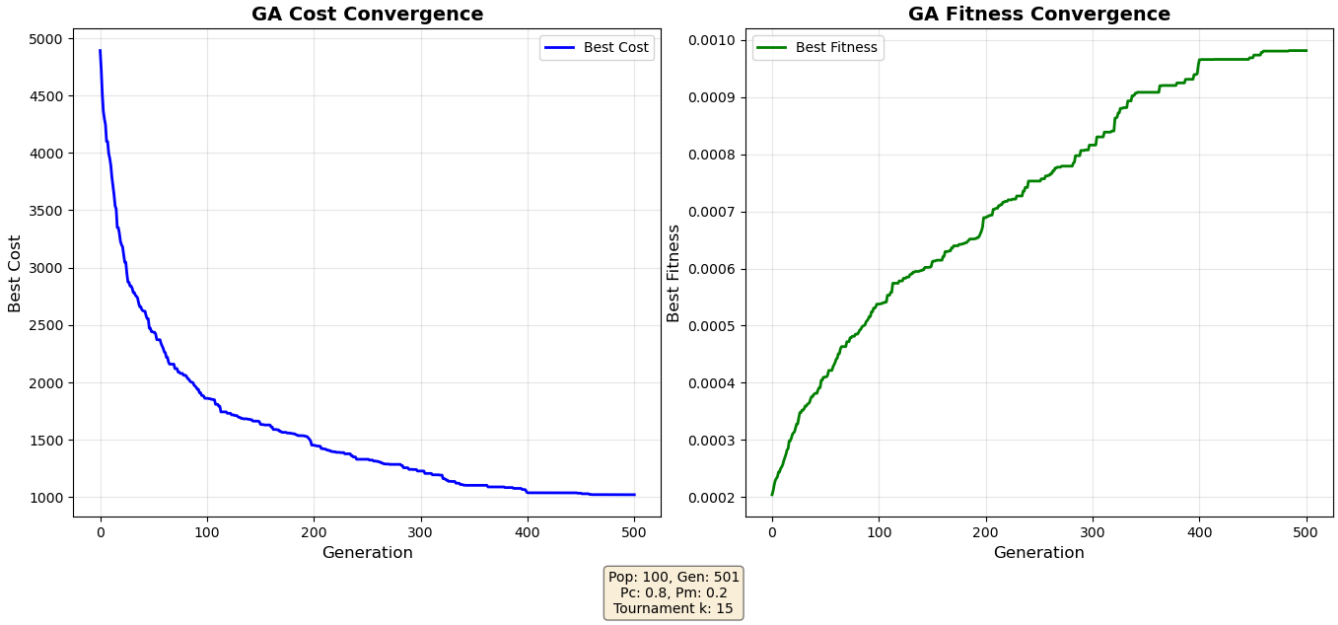
6.2.3 Genetic Algorithm - GA

Xét *TSP* $m = 100$ thành phố, các tham số mặc định cho *GA* là *generation* $= 500$, $N = 100$, $p_{crossover} = 0.8$, $p_{mutation} = 0.2$, $k = 5$ và hằng số $\rho = 2$. Lần lượt thay đổi một tham số và cố định các tham số còn lại để đo độ nhạy (sự ảnh hưởng) của tham số đối với lời giải:

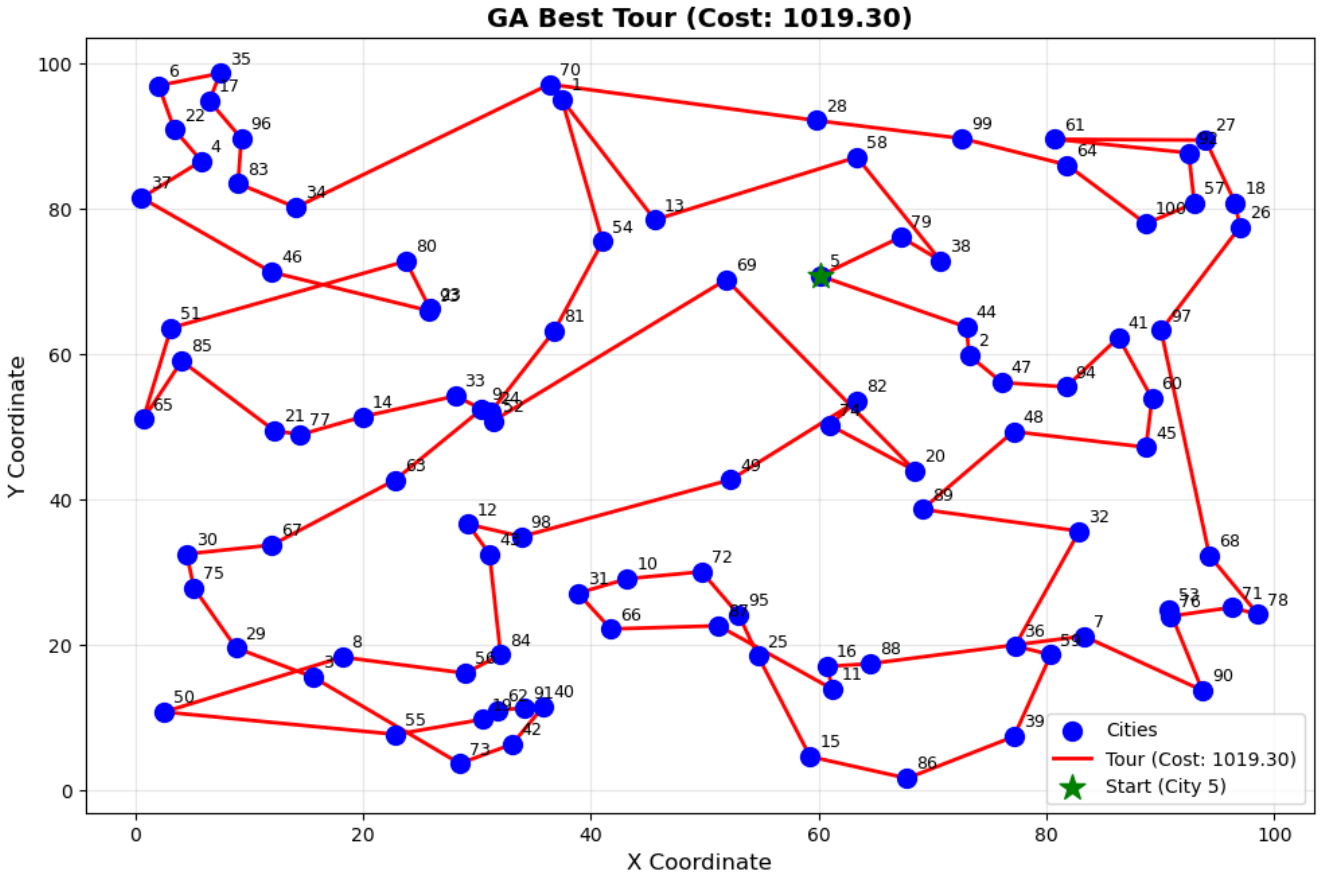


Hình 29: Độ nhạy tham số với thuật toán GA

Biểu đồ cho thấy các tham số $N = 100$, $p_{crossover} = 0.8$, $p_{mutation} = 0.2$, $k = 15$ là tối ưu nhất. Với k, càng tăng thì lời giải càng tốt (đạt giới hạn ở $k_{max} = 15$ ở thử nghiệm này). Ta sẽ sử dụng các tham số trên để chạy *GA*.



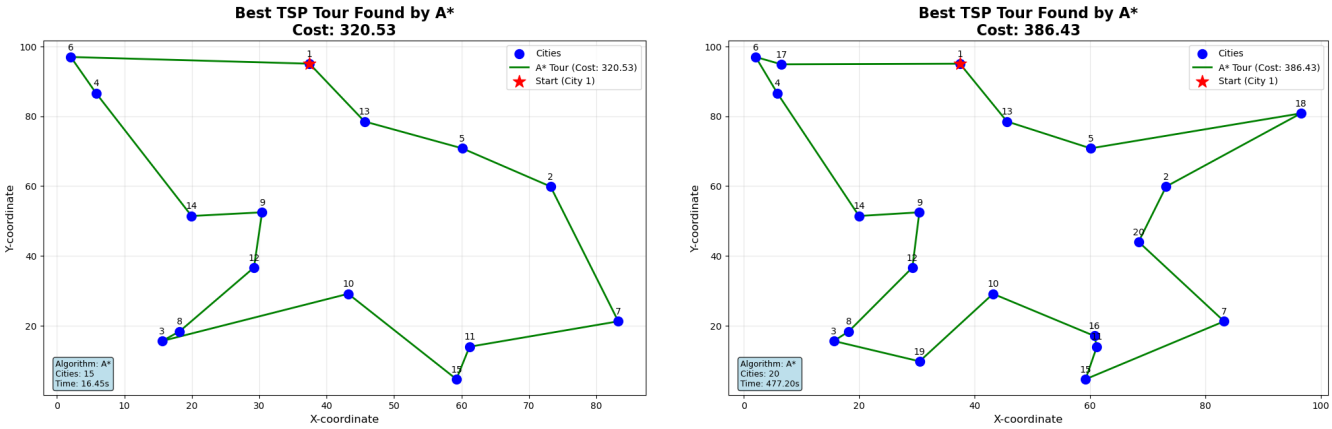
Hình 30: Tốc độ hội tụ và hàm thích nghi của GA



Hình 31: Đường đi Hamilton tìm bởi GA cho TSP

6.2.4 A* search

Ta sử dụng $A*$ để giải quyết *TSP* với số thành phố lần lượt là $m = 15$ và $m = 20$. Có thể thấy rằng chất lượng và tính ổn định của thuật toán $A*$ là rất tốt, nhưng khả năng mở rộng với m lớn là gần như không có vì độ phức tạp thời gian quá lớn.



(a) m = 15 (16 giây)

(b) m = 20 (477 giây)

Hình 32: Đường đi Hamilton tìm bởi A* cho TSP

6.3 Kết luận

Với *TSP* dùng các giải thuật trí tuệ bầy đàn *ACO*, *TSP* và thuật toán truyền thống *GA*, $A*$, ta có các kết luận sau:

- **Chất lượng lời giải:** $A*$ có lời giải tốt nhưng độ phức tạp thời gian lớn. *ACO* có lời giải tối ưu nhất. *GA* thường có lời giải tốt hơn *PSO*.
- **Tốc độ hội tụ:** *ACO* có tốc độ hội tụ rất nhanh, sau đó đến *GA*. *PSO* hội tụ khá chậm.
- **Độ phức tạp thời gian:** *GA* có tốc độ nhanh nhất. *ACO* nhanh hơn *PSO* ở đa số trường hợp. $A*$ rất chậm khi số thành phố lớn ($m \geq 25$).
- **Tính ổn định của thuật toán:** *ACO* và $A*$ tương đối ổn định. *GA* và *PSO* có tính ổn định kém hơn.
- **Khả năng mở rộng:** Để giải quyết *TSP* với m lớn, *GA* và đặc biệt là *ACO* cực kì phù hợp vì lời giải vẫn tốt và thời gian tính toán nhanh. *PSO* chậm, lời giải kém hơn. $A*$ không có khả năng giải quyết với m lớn.
- **Độ nhạy tham số:** *ACO*, *TSP* và *GA* đều biến động lớn về lời giải khi có sự thay đổi về tham số.

Nhìn chung, *ACO* phù hợp nhất để giải quyết *TSP*. *PSO* ban đầu là thuật toán để giải quyết các bài toán tối ưu liên tục, nhưng vẫn có thể tùy chỉnh để giải quyết bài toán tối ưu rời rạc. *GA* có tốc độ tốt và lời giải ổn. $A*$ lời giải tốt nhất nhưng độ phức tạp thời gian rất lớn.

7 Kết luận, đánh giá

7.1 Kết luận

Đồ án đã thực hiện thành công mục tiêu đề ra là triển khai, phân tích và so sánh một cách toàn diện các thuật toán trí tuệ bầy đàn và thuật toán truyền thống trên hai loại bài toán tối ưu hóa: liên tục và rời rạc. Trí tuệ bầy đàn đã chứng tỏ là một phương pháp mạnh mẽ, linh hoạt và hiệu quả cho các bài toán tối ưu hóa phức tạp. Qua quá trình thực nghiệm và phân tích kết quả, nhóm đã rút ra được những kết luận quan trọng sau:

- Đối với bài toán tối ưu hóa liên tục (Hàm *Sphere* và *Rastrigin*):
  - Các thuật toán trí tuệ bầy đàn (*Cuckoo Search - CS*, *Firefly Algorithm - FA*, *Artificial Bee Colony - ABC*) thể hiện sự vượt trội rõ rệt so với các thuật toán truyền thống (*Hill Climbing - HC*, *Simulated Annealing - SA*) về cả tốc độ hội tụ và chất lượng lời giải cuối cùng. *CS* và *FA* cho thấy khả năng thoát khỏi các điểm tối ưu cục bộ rất tốt, nhanh chóng tìm đến vùng lân cận của tối ưu toàn cục.
  - *ABC* chứng tỏ khả năng mở rộng tốt nhất khi số chiều của bài toán tăng lên, tuy nhiên lại kém ổn định hơn so với *CS* và *FA* trên hàm phức tạp như *Rastrigin*.
- Đối với bài toán tối ưu hóa rời rạc (*Travelling Salesman Problem - TSP*):
  - Tối ưu hóa Đàn kiến (*Ant Colony Optimization - ACO*) tỏ ra là thuật toán hiệu quả và phù hợp nhất để giải quyết *TSP*. *ACO* không chỉ cho ra lời giải có chất lượng tốt nhất (chi phí thấp nhất) mà còn thể hiện tính ổn định cao qua nhiều lần chạy.
  - Giải thuật Di truyền (*Genetic Algorithm - GA*) cung cấp một sự cân bằng tốt giữa tốc độ và chất lượng, cho lời giải chấp nhận được trong thời gian nhanh hơn đáng kể so với các thuật toán khác.
  - Tối ưu hóa Bầy hạt (*Particle Swarm Optimization - PSO*), vốn được thiết kế cho không gian liên tục, khi được điều chỉnh cho bài toán *TSP* cho thấy hiệu quả kém hơn, hội tụ chậm và chất lượng lời giải không cao bằng *ACO* và *GA*.
  - Thuật toán A* (*A* Search*) tìm ra được lời giải tối ưu nhưng chi phí tính toán tăng theo hàm mũ với số lượng thành phố, không khả thi cho các bài toán có kích thước lớn.

7.2 Đánh giá

Đối với đồ án này, nhóm tự đánh giá **hoàn thành tốt và đầy đủ** các yêu cầu đã được đặt ra. Chi tiết như sau:

- Các giải thuật trí tuệ bầy đàn, thuật toán truyền thống và bài toán được trình bày chi tiết, kĩ lưỡng với cơ sở khoa học vững chắc từ các bài báo.
- Code cho thuật toán được cài đặt kĩ lưỡng, đúng logic và định dạng.
- Có thực nghiệm, trực quan hóa và so sánh các giải thuật.

Tài liệu

[1] Anh Khoa, Hoang Trung, and Duc Tai. github/anhKhoa585/AI-Fundamentals-Proj1. https://github.com/anhKhoa585/AI-Fundamentals-Proj1, 2025.

[2] Anh Khoa, Hoang Trung, and Duc Tai. Demo swarm intelligence algorithm for optimization problems. https://www.youtube.com/watch?v=KFdTRbJd9w0, 2025.

[3] Eugene L Lawler. The traveling salesman problem: a guided tour of combinatorial optimization. *Wiley-Interscience Series in Discrete Mathematics*, 1985.

[4] Gilbert Laporte. The traveling salesman problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(2):231–247, 1992.

[5] Xin-She Yang and Suash Deb. Cuckoo search via levy flights. In *Proc. of World Congress on Nature & Biologically Inspired Computing (NaBic 2009)*, pages 210–214, USA, 2009. IEEE Publications.

[6] An idea based on honey bee swarm for numerical optimization. 2007.

[7] Xin-She Yang. Firefly algorithms for multimodal optimization. In Osamu Watanabe and Thomas Zeugmann, editors, *Stochastic Algorithms: Foundations and Applications*, pages 169–178, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

[8] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. Ant colony optimization. *IEEE computational intelligence magazine*, 1(4):28–39, 2007.

[9] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, volume 4, pages 1942–1948. ieee, 1995.

[10] Yuhui Shi and Russell Eberhart. A modified particle swarm optimizer. In *1998 IEEE international conference on evolutionary computation proceedings. IEEE world congress on computational intelligence (Cat. No. 98TH8360)*, pages 69–73. Ieee, 1998.

[11] Xiaohu H Shi, Yanchun Chun Liang, Heow Pueh Lee, C Lu, and QX Wang. Particle swarm optimization-based algorithms for tsp and generalized tsp. *Information processing letters*, 103(5):169–176, 2007.

[12] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.

[13] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[14] John H Holland. *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press, 1992.

[15] Noraini Mohd Razali, John Geraghty, et al. Genetic algorithm performance with different selection strategies in solving tsp. In *Proceedings of the world congress on engineering*, volume 2, pages 1–6. International Association of Engineers Hong Kong, China, 2011.